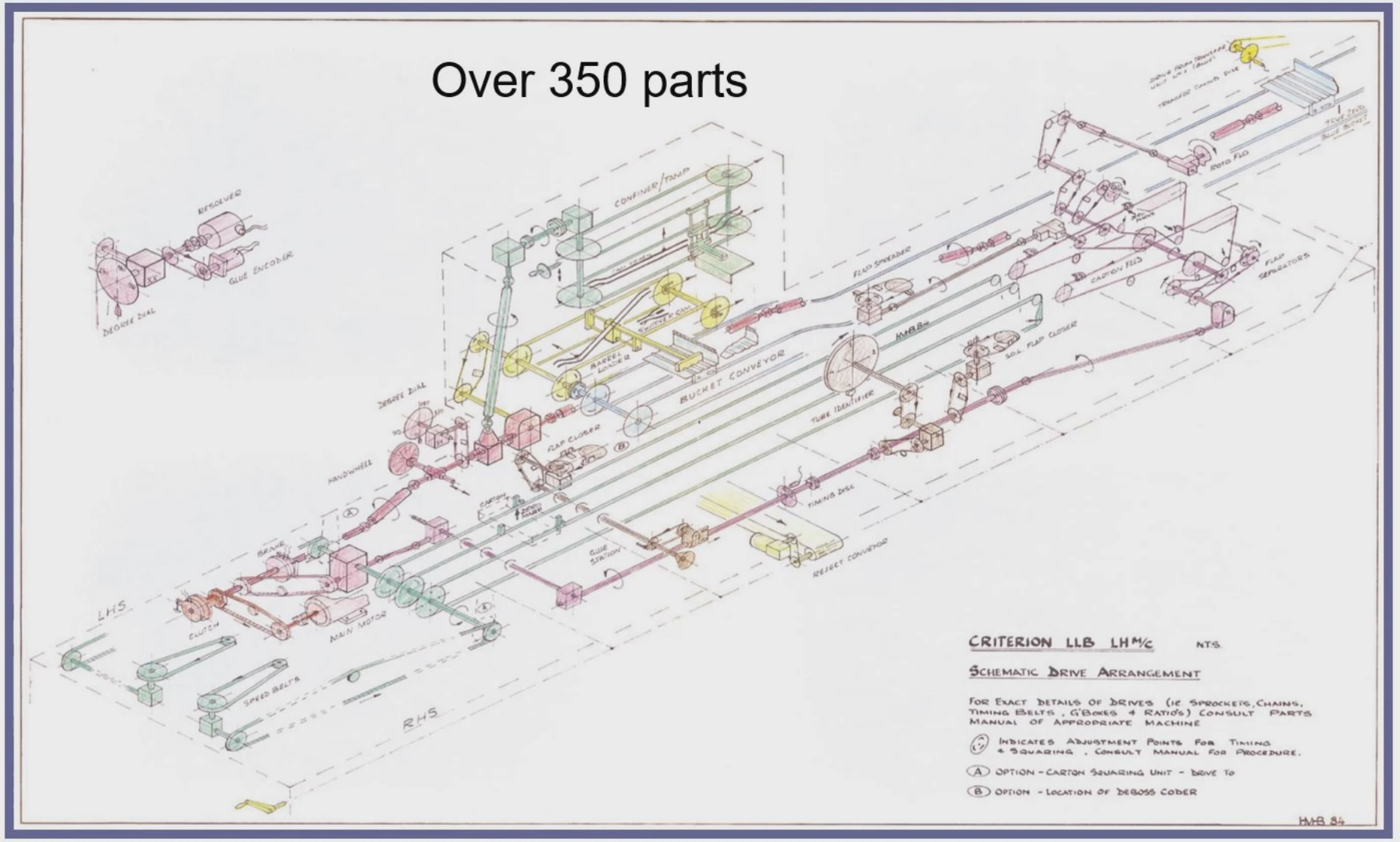
PLCopen Motion Control

Making the world go round

https://youtu.be/2u9jKBaIeW0

Traditional Mechanical Design

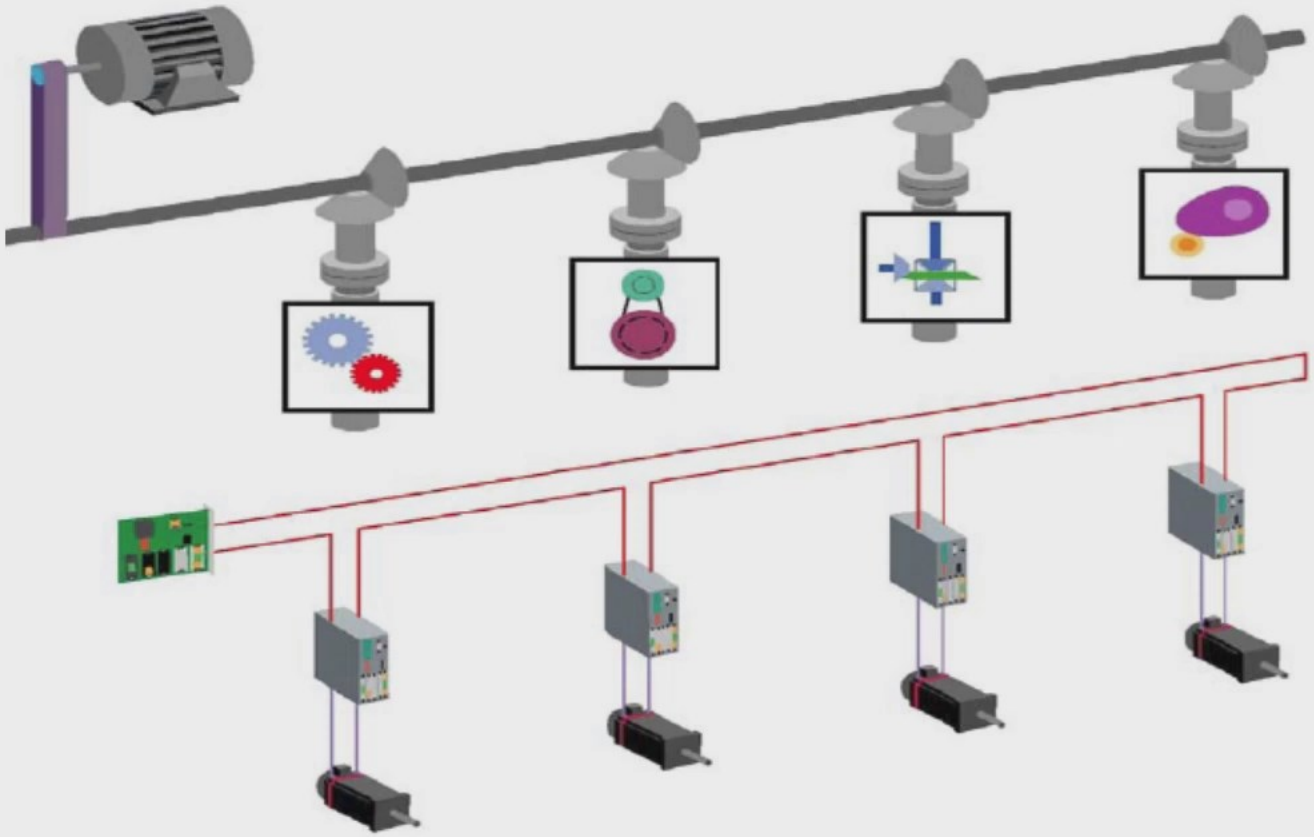


Drawbacks

- Too many parts, expensive to build
- High level of maintenance (usage of oil)
- Non-predictive maintenance
- Less flexible (changeovers)
- More waste in production
- Difficult to clean
- Contamination of packaged product possible

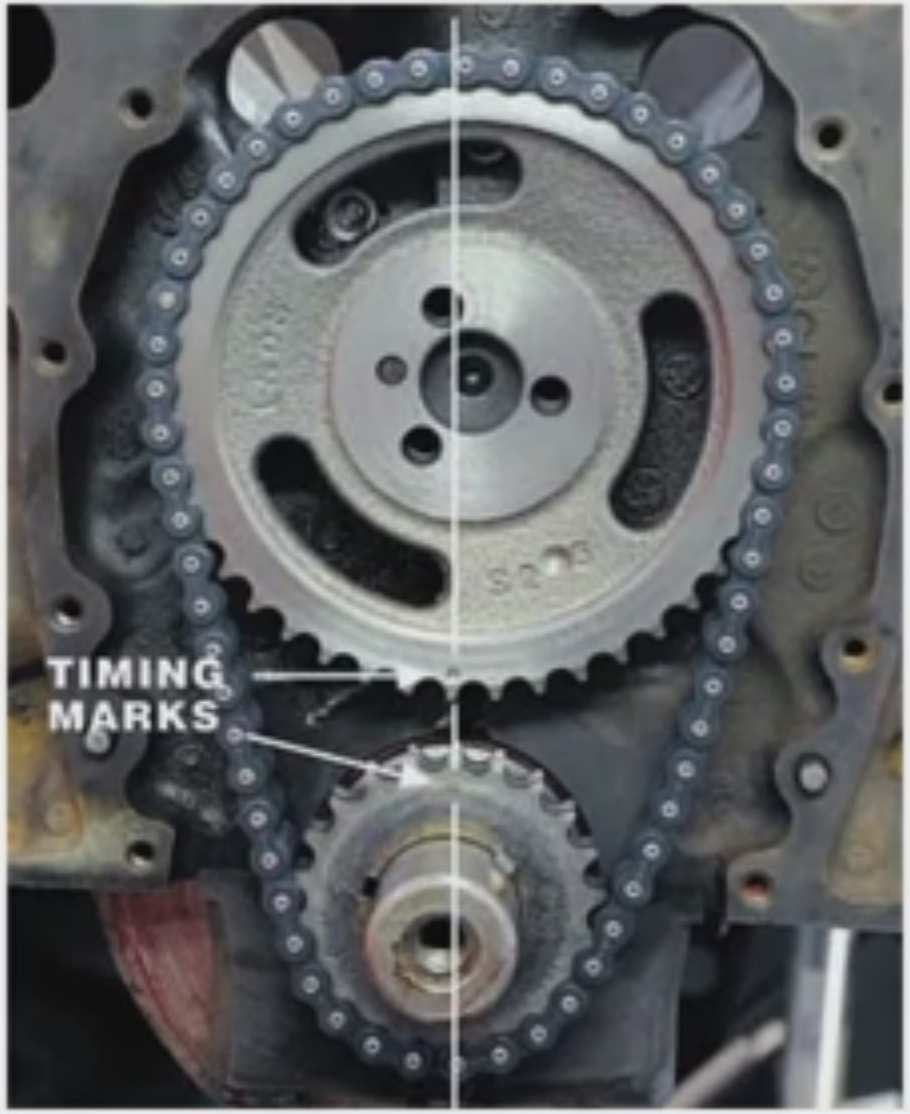
Future proof: mechatronic solutions

Mechanical solution.



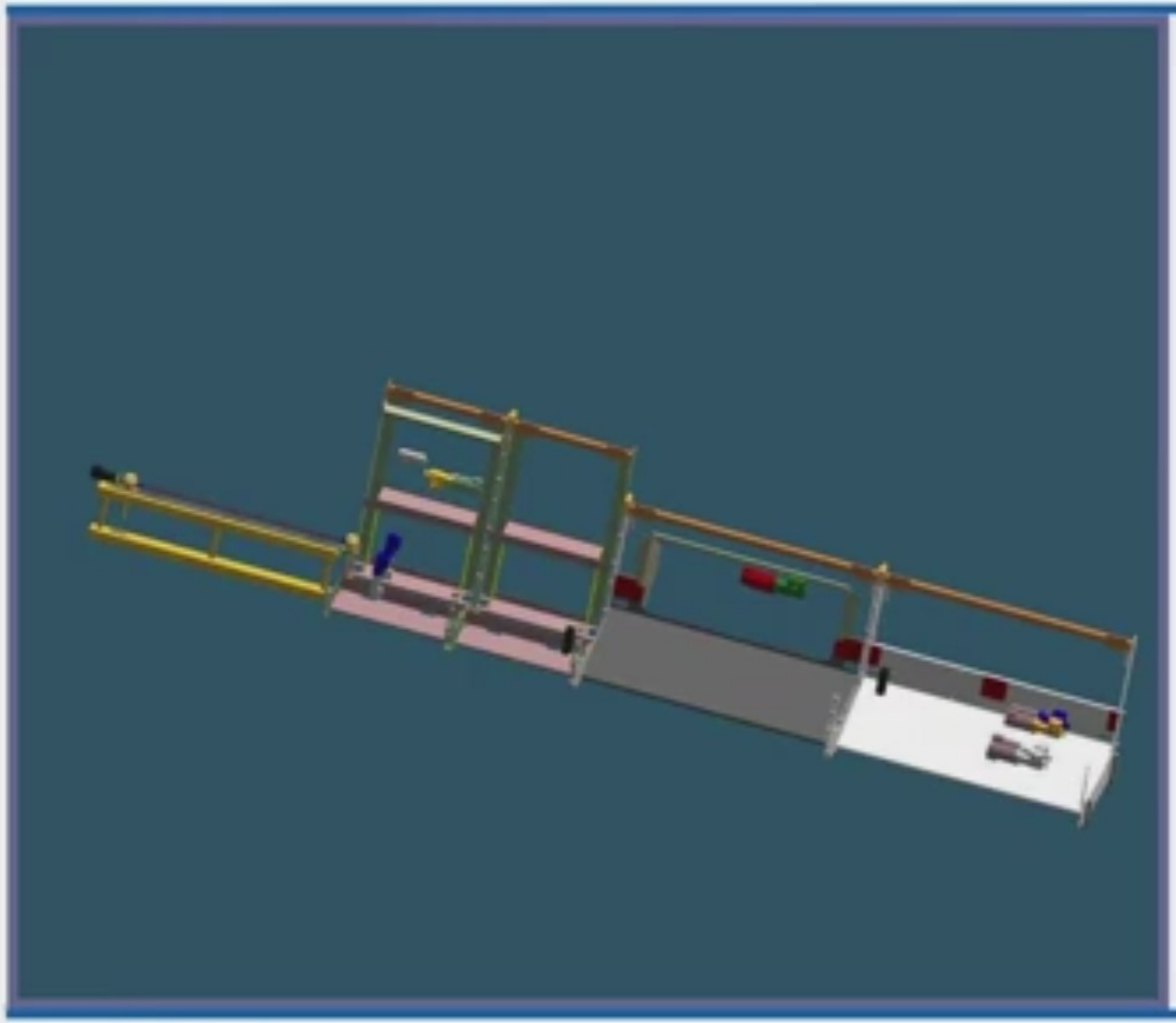
Control solution

Mechatronics example



Mechanical connections are replaced by software commands

Solution: Multi Axis Servo Drives



Major part count reduction

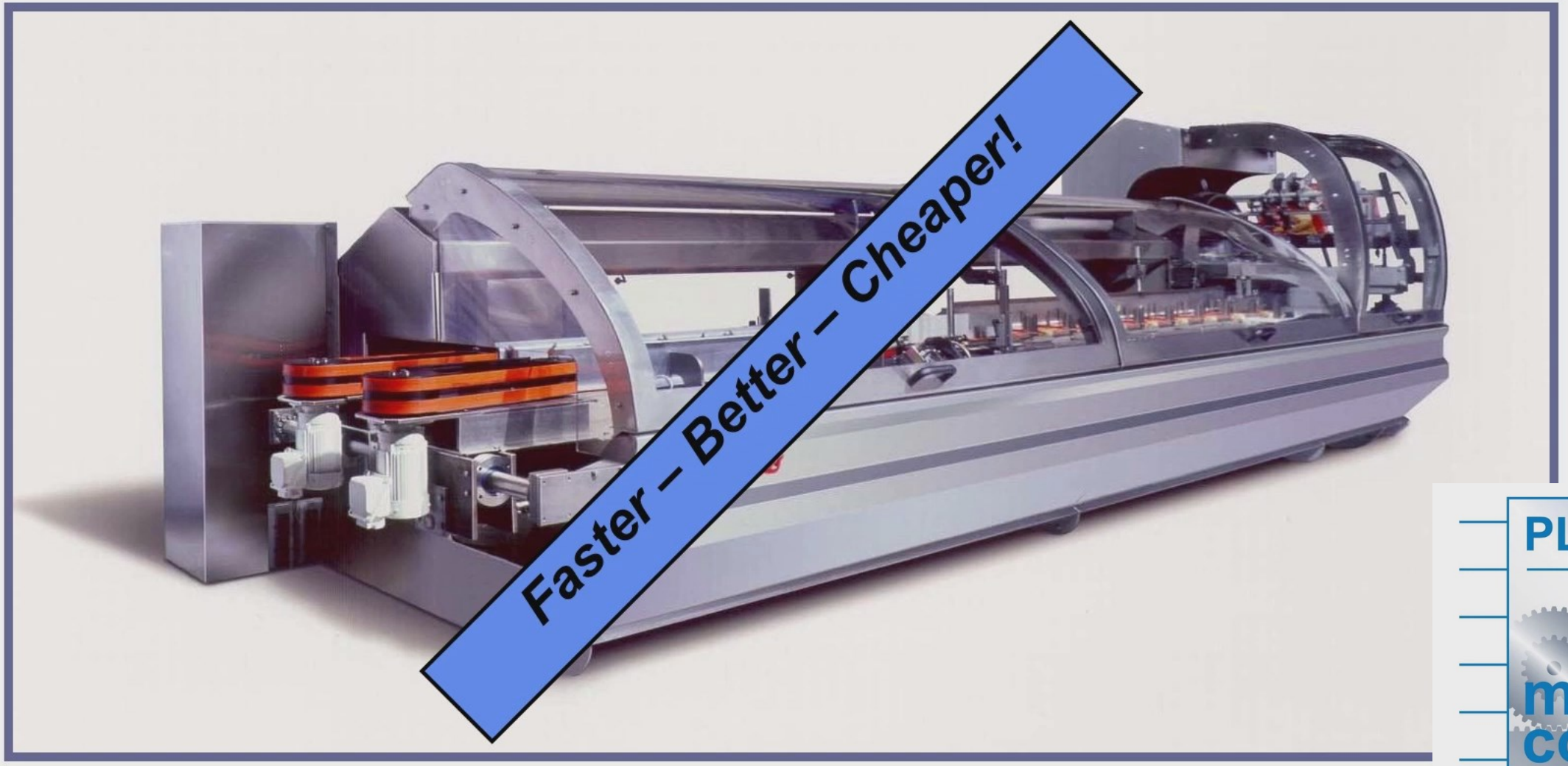
Pulleys	- 45 to 0
Belts	- 15 to 0
Drive sprockets	- 15 to 0
Spline shafts	- 2 to 0
Gearboxes	- 16 to 10
Motors	- 1 to 10
Bearings	- 18 to 3
Line shafts	- 6 to 0
Total	- 118 to 23

(81

PLCopen® motion control

In praxis : from this ….

.. to this



PLCopen is supporting this transition

The controller needs software

Software needs re-usability via abstraction

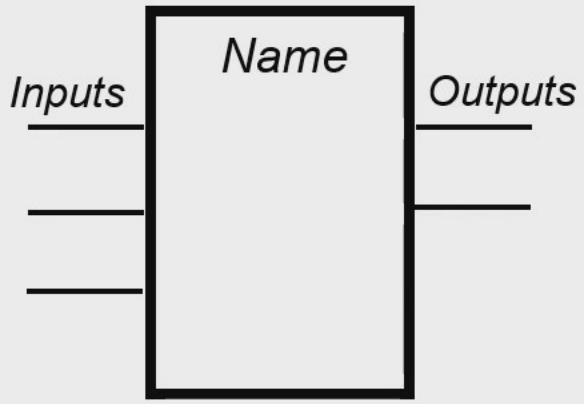
Software needs standards

PLCopen helps

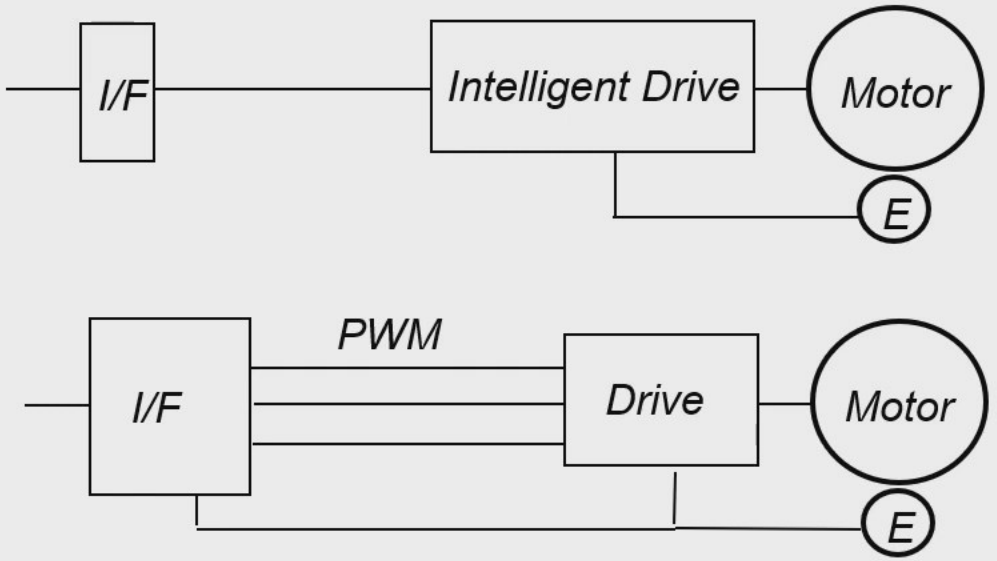
Abstraction / HW Independence via FBs

Software View

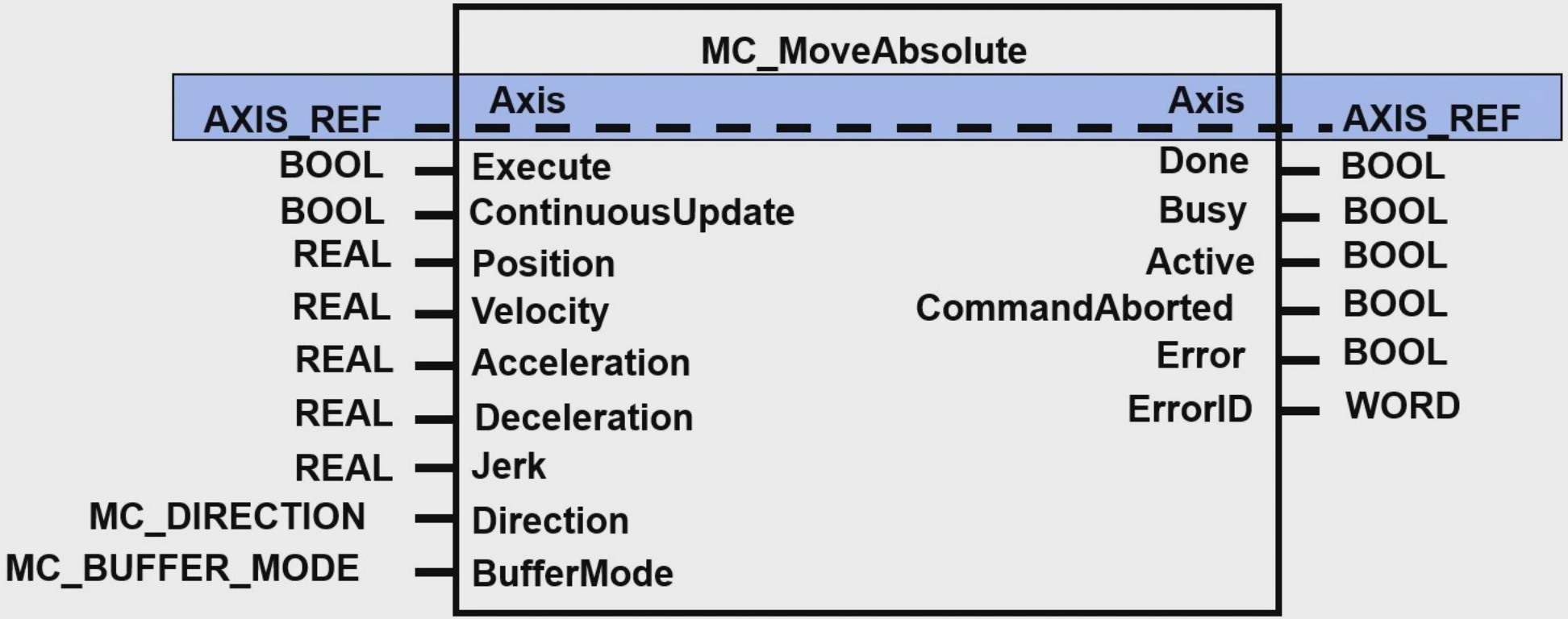
Encapsulation / Information Hiding



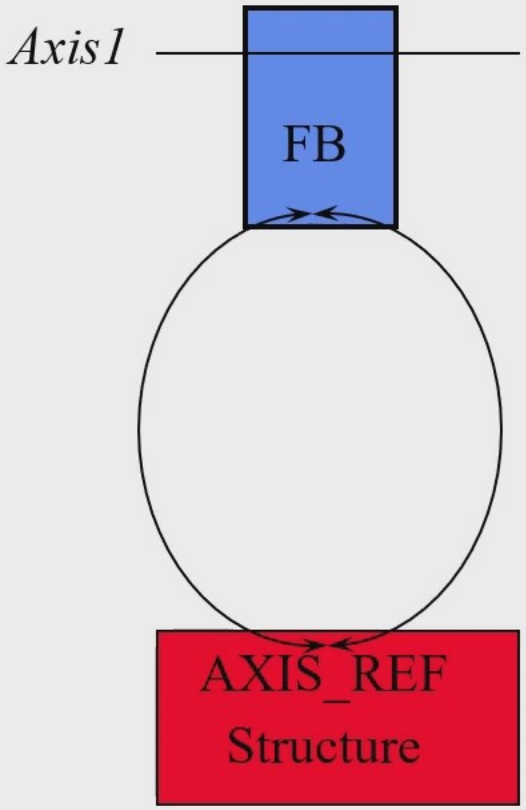
Hardware View



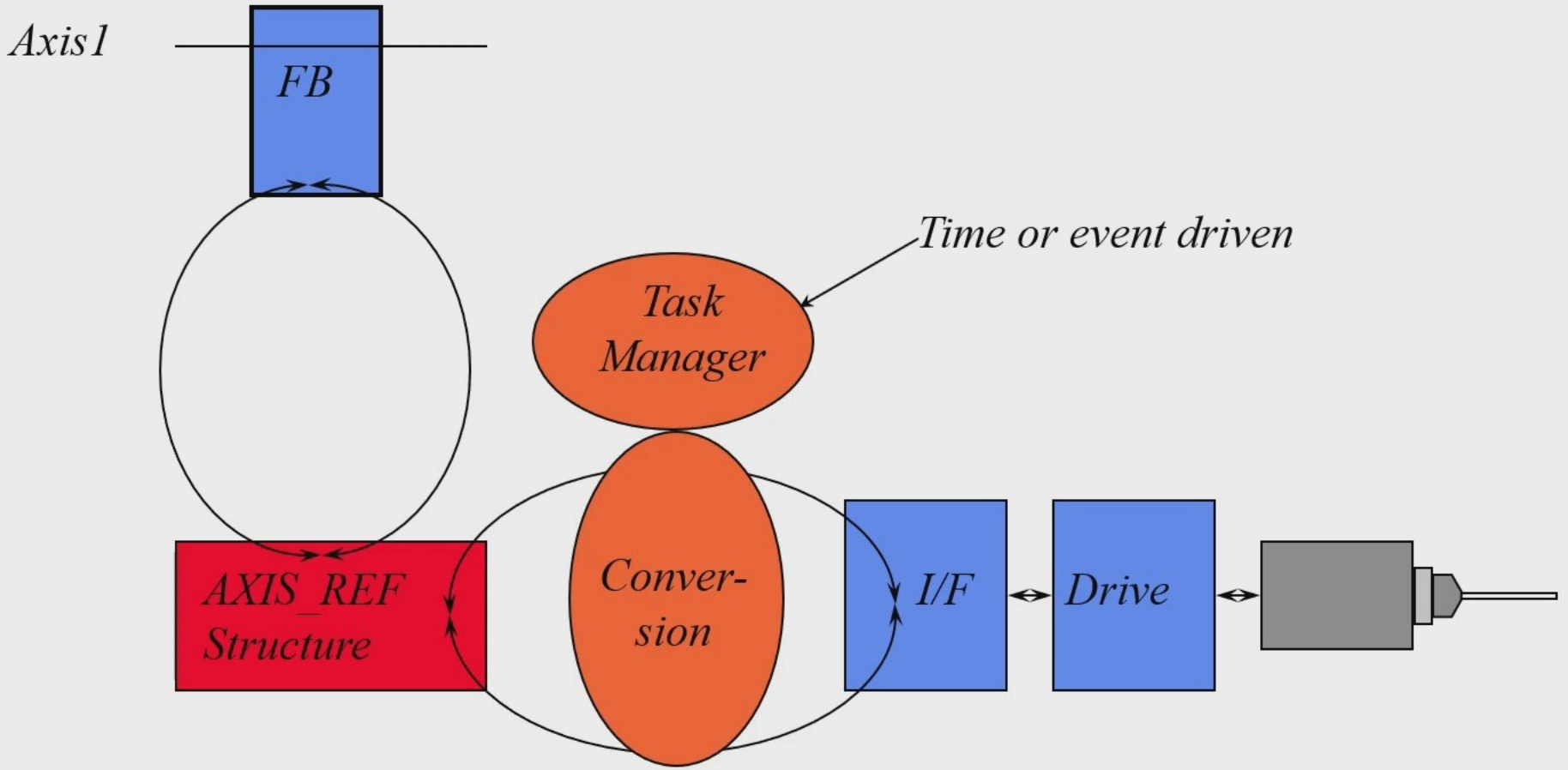
Abstraction via Function Blocks



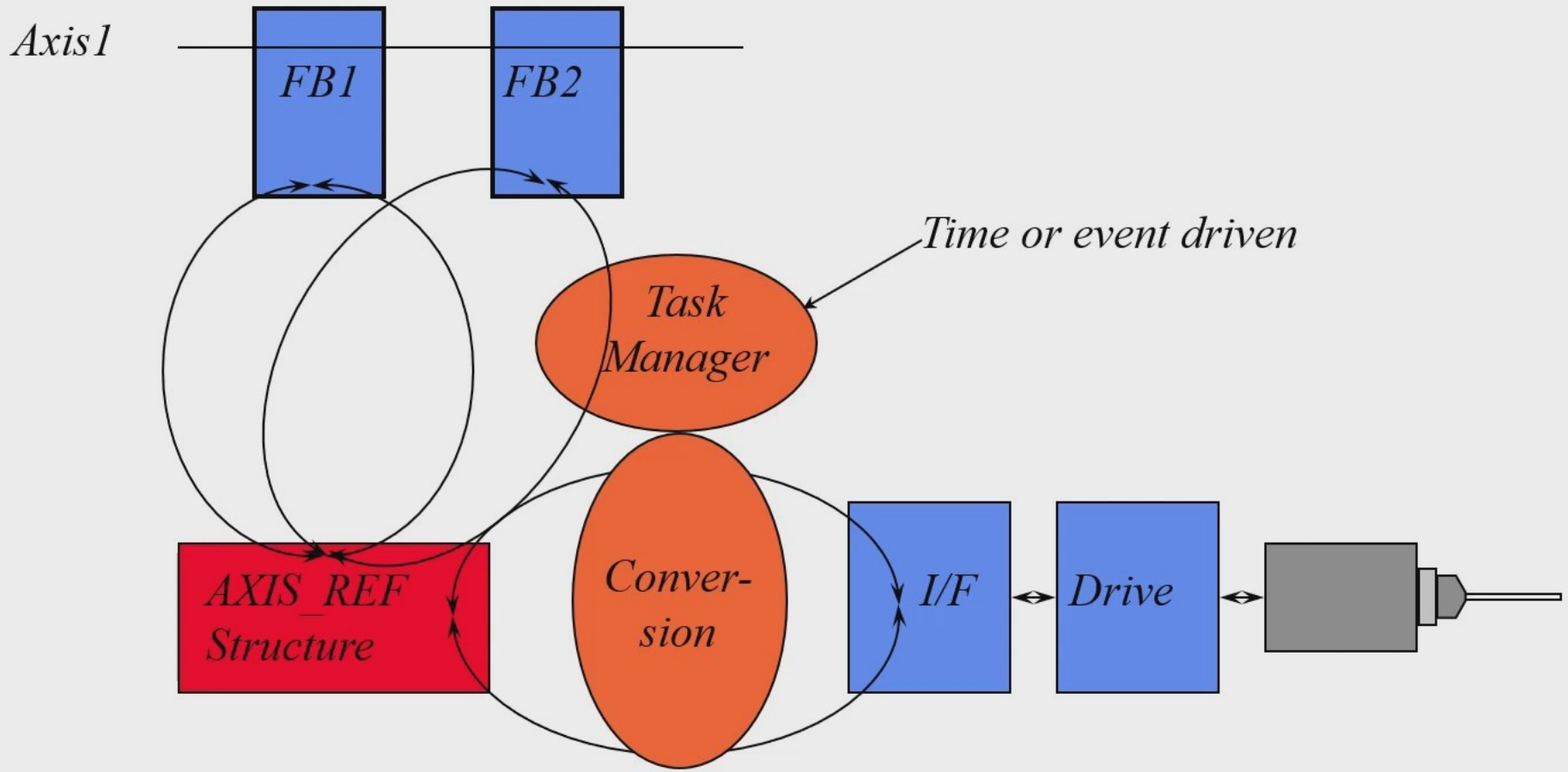
AXIS_REF as Var_In_Out



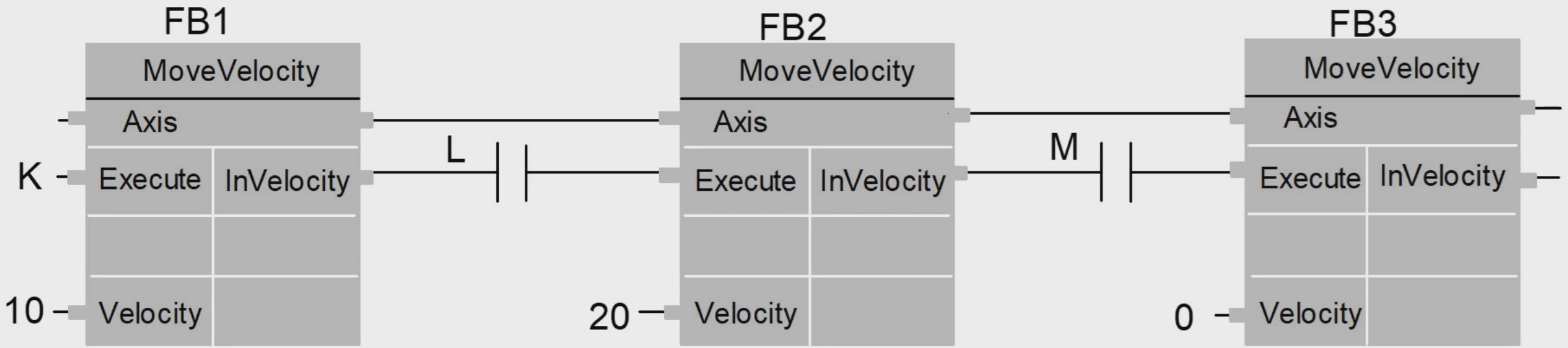
Abstraction with one FB …



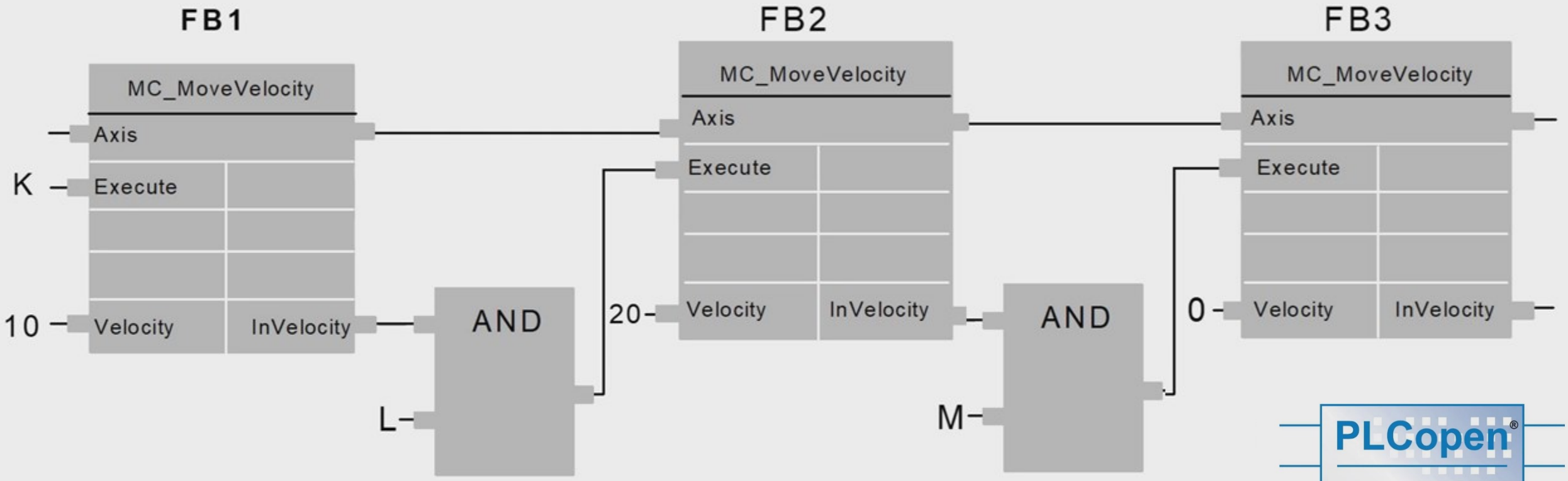
… and with 2 FBs



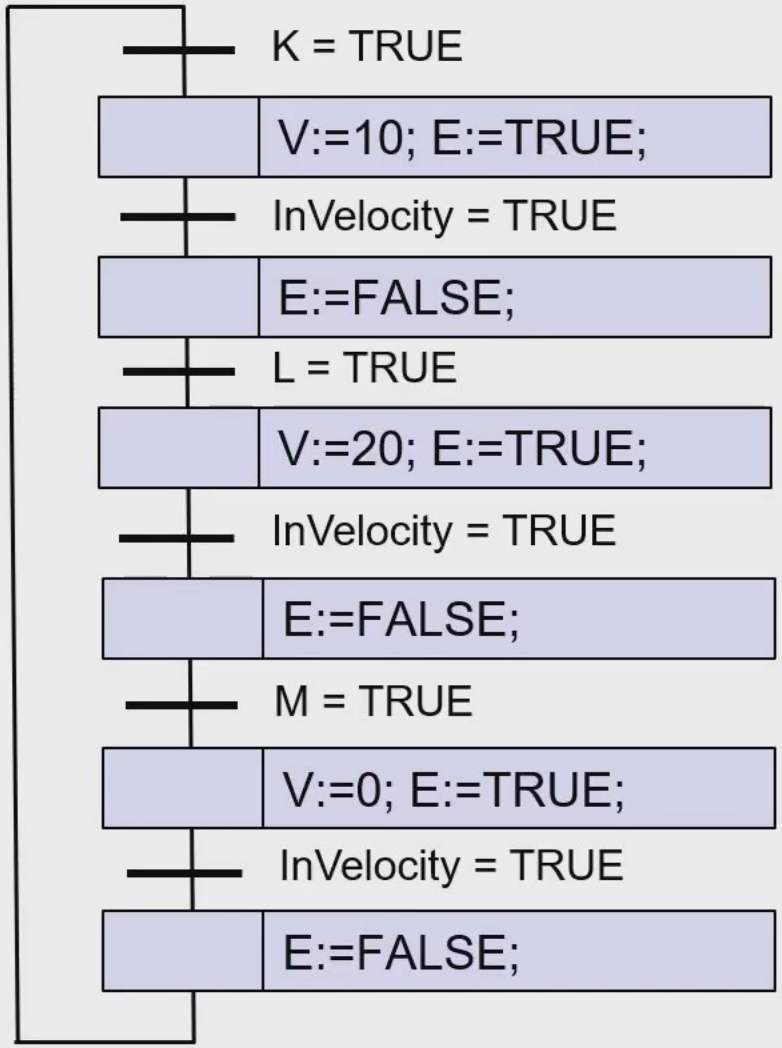
Example – Multiple FBs on 1 axis - LD

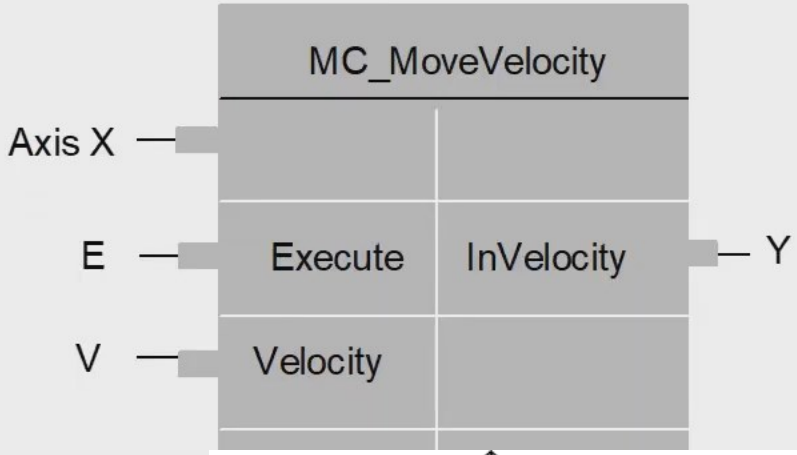


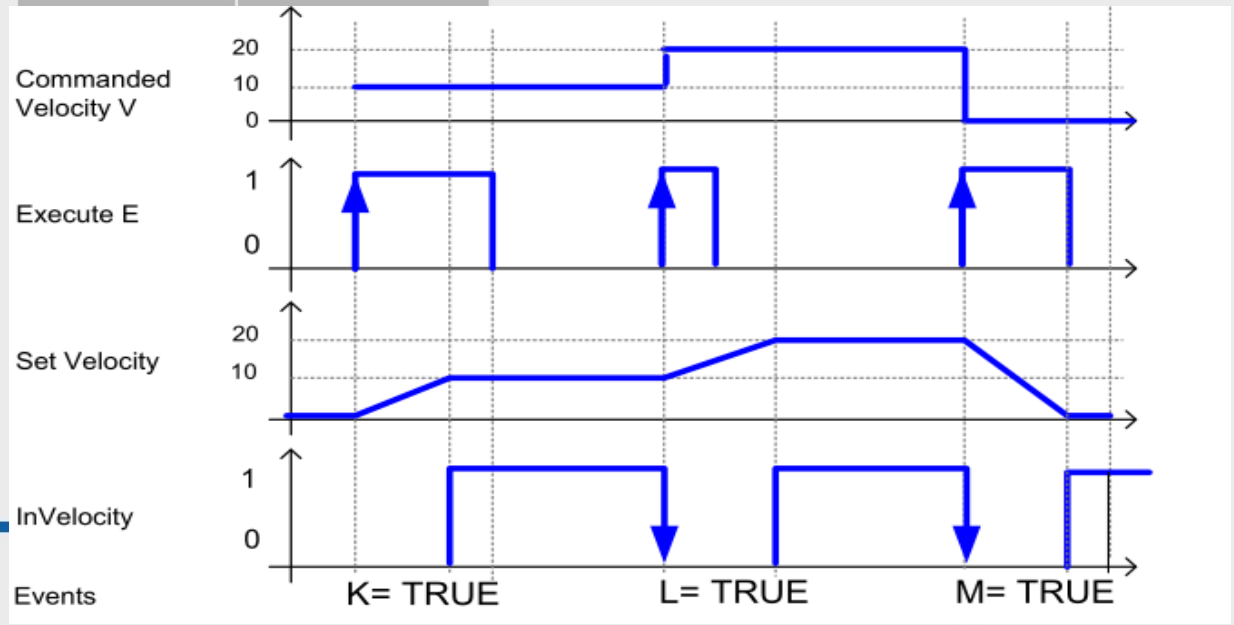
Example – Multiple FBs on 1 axis - FBD



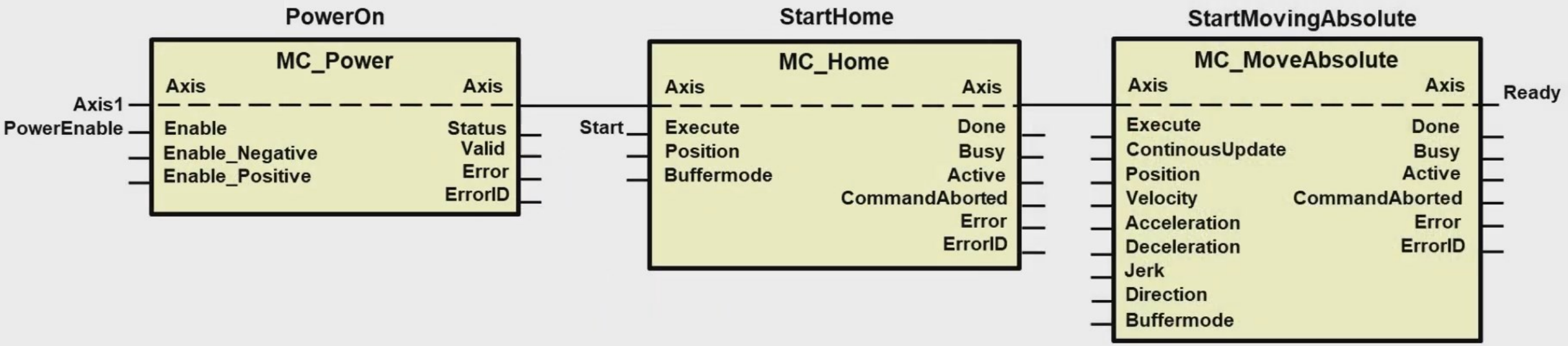
Example with SFC



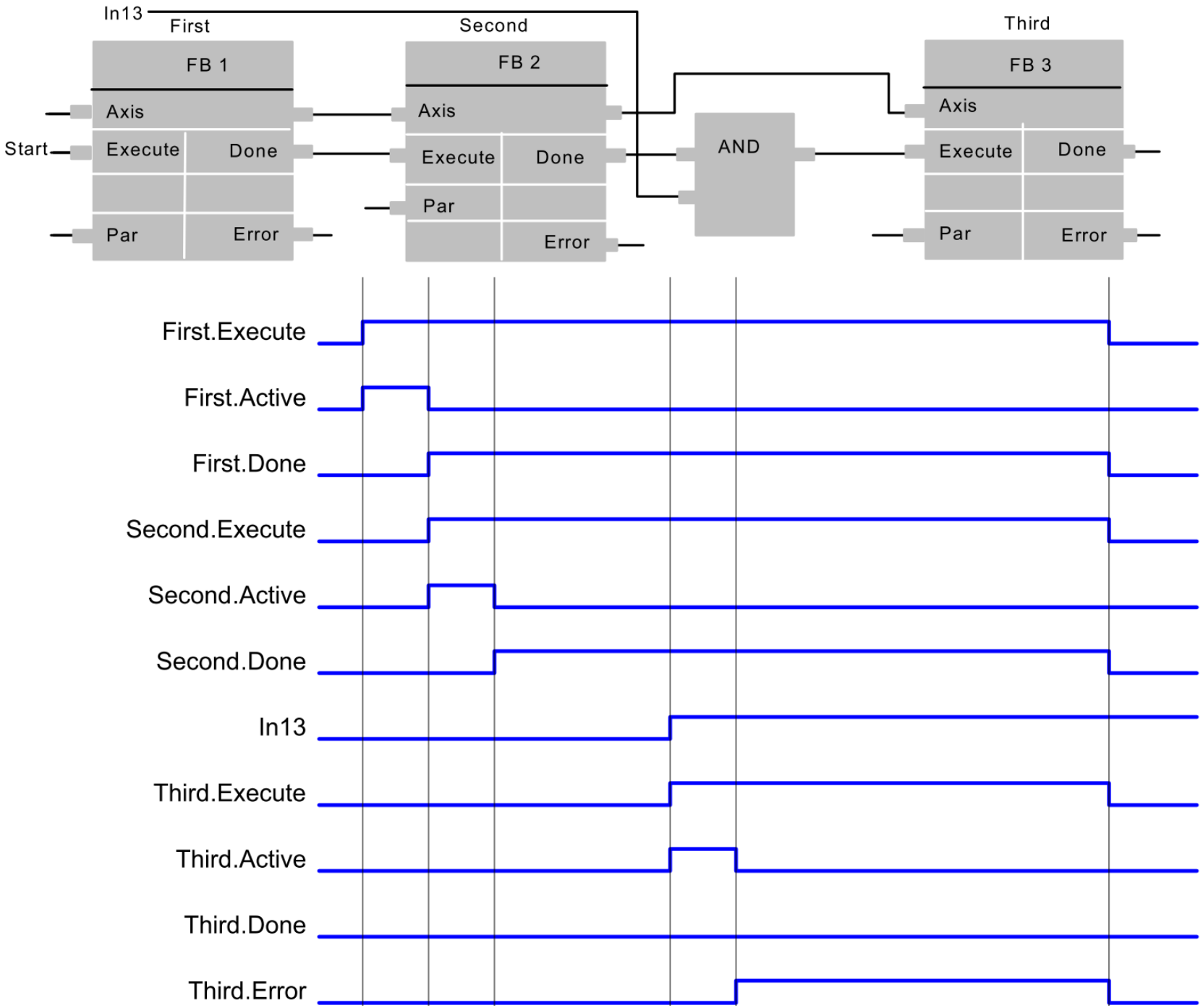


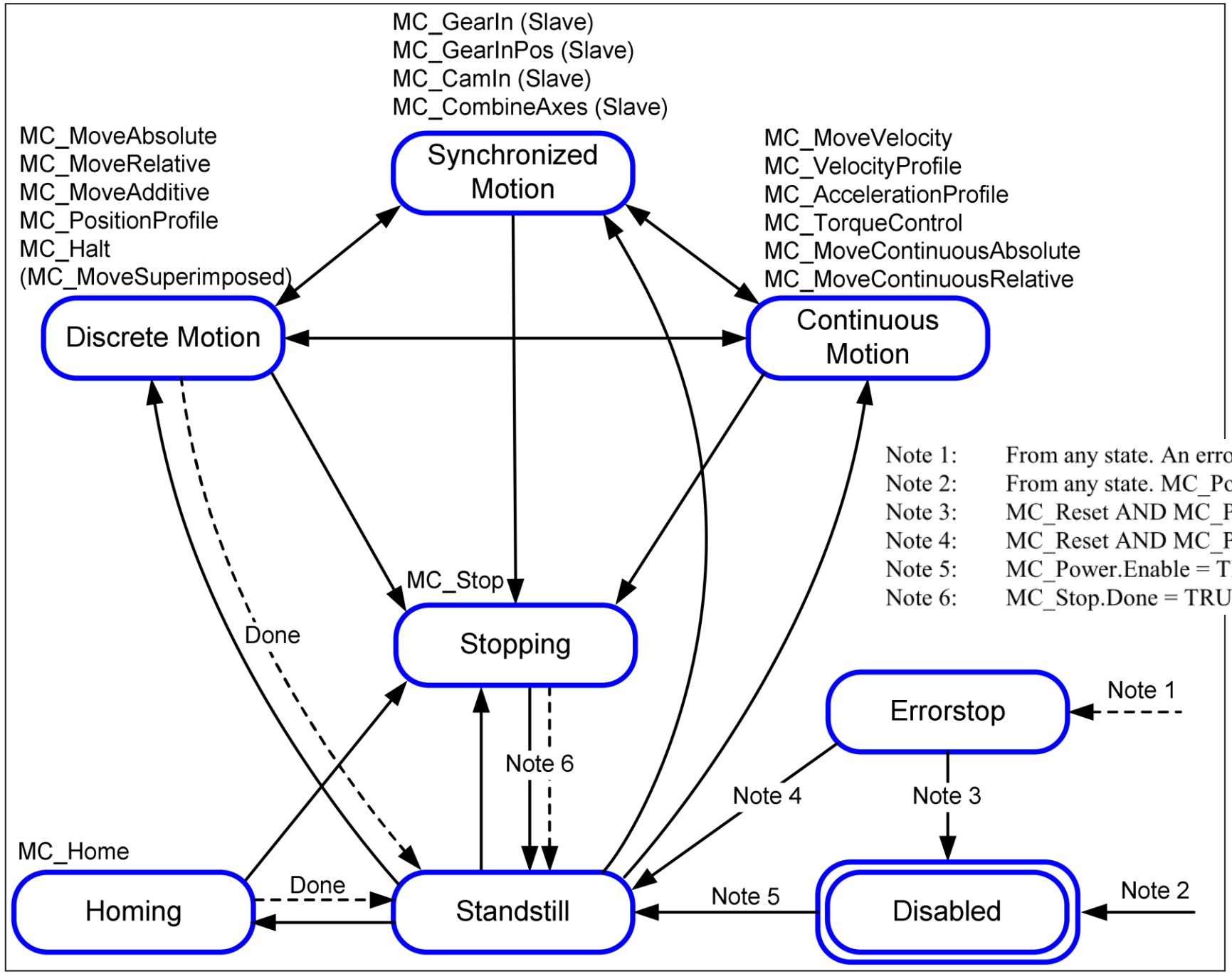


Start-up procedure



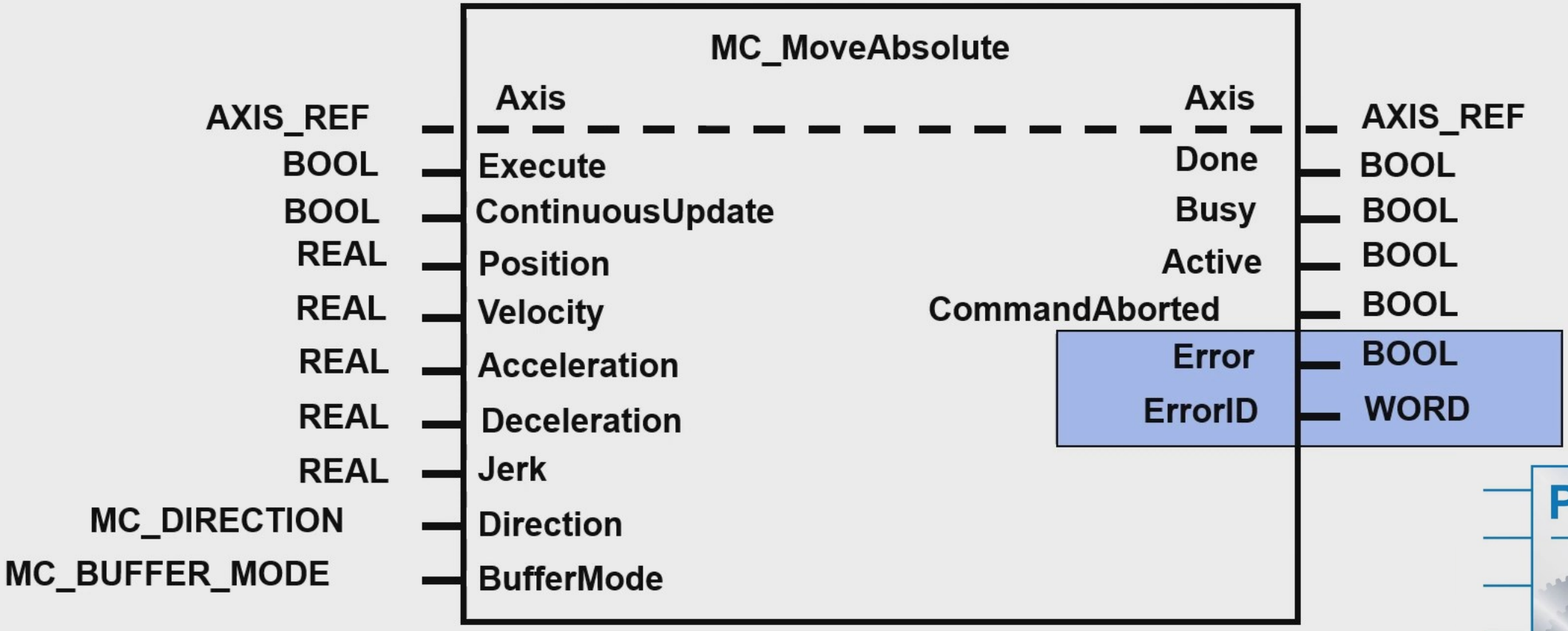
Independent of the architecture


In13
First
Second
Third
FB 1
FB 2
FB 3
Axis
Execute
Done
Par
Error
Start
AND
First.Execute
First.Active
First.Done
Second.Execute
Second.Active
Second.Done
In13
Third.Execute
Third.Active
Third.Done
Third.Error

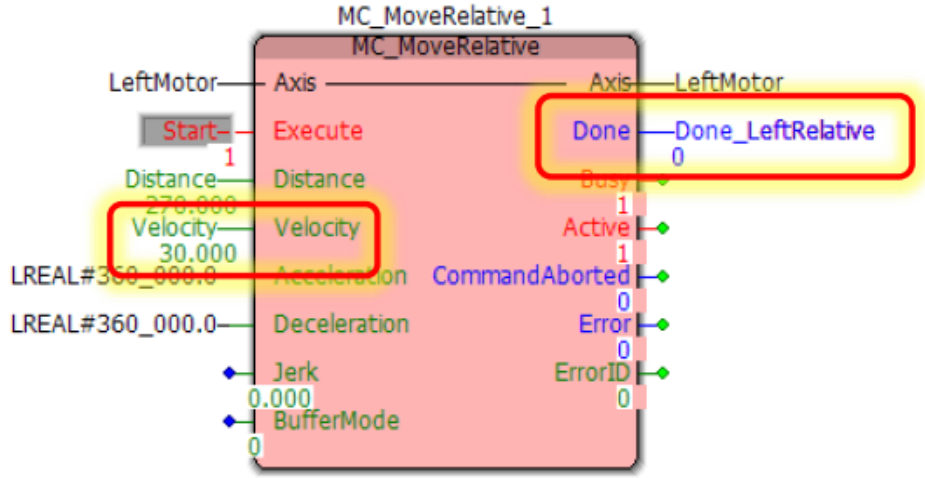


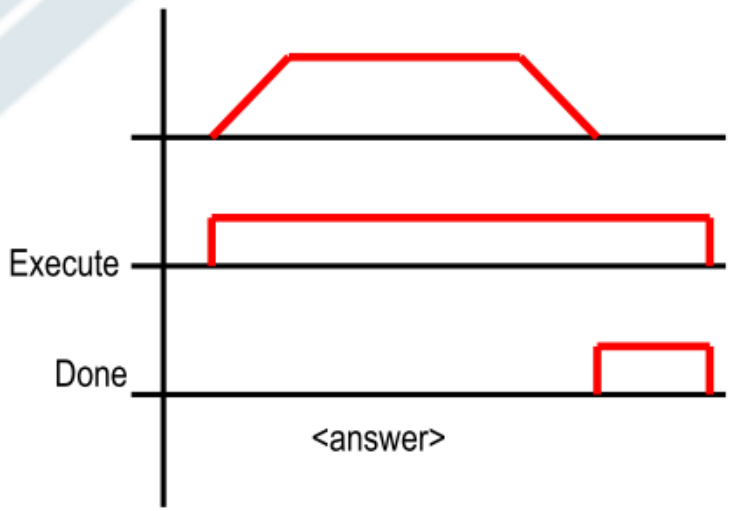
Note 1: From any state. An error in the axis occurred.
Note 2: From any state. MC_Power.Enable = FALSE and there is no error in the axis.
Note 3: MC_Reset AND MC_Power.Status = FALSE
Note 4: MC_Reset AND MC_Power.Status = TRUE AND MC_Power.Enable = TRUE
Note 5: MC_Power.Enable = TRUE AND MC_Power.Status = TRUE
Note 6: MC_Stop.Done = TRUE AND MC_Stop.Execute = FALSE

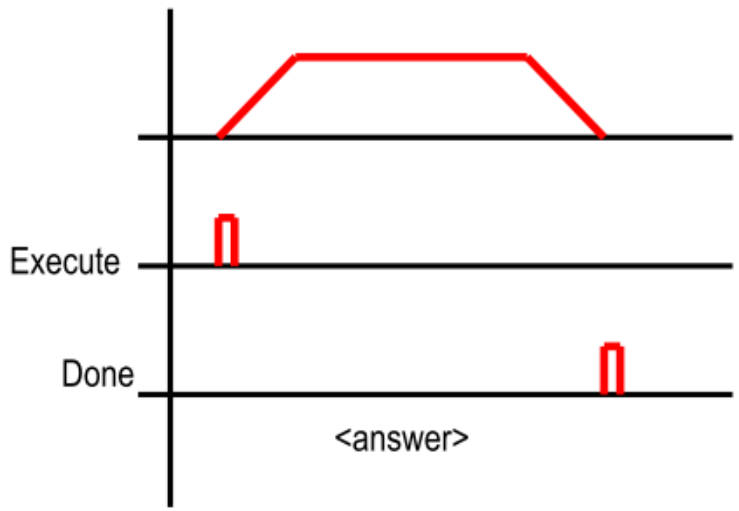
Error Handling



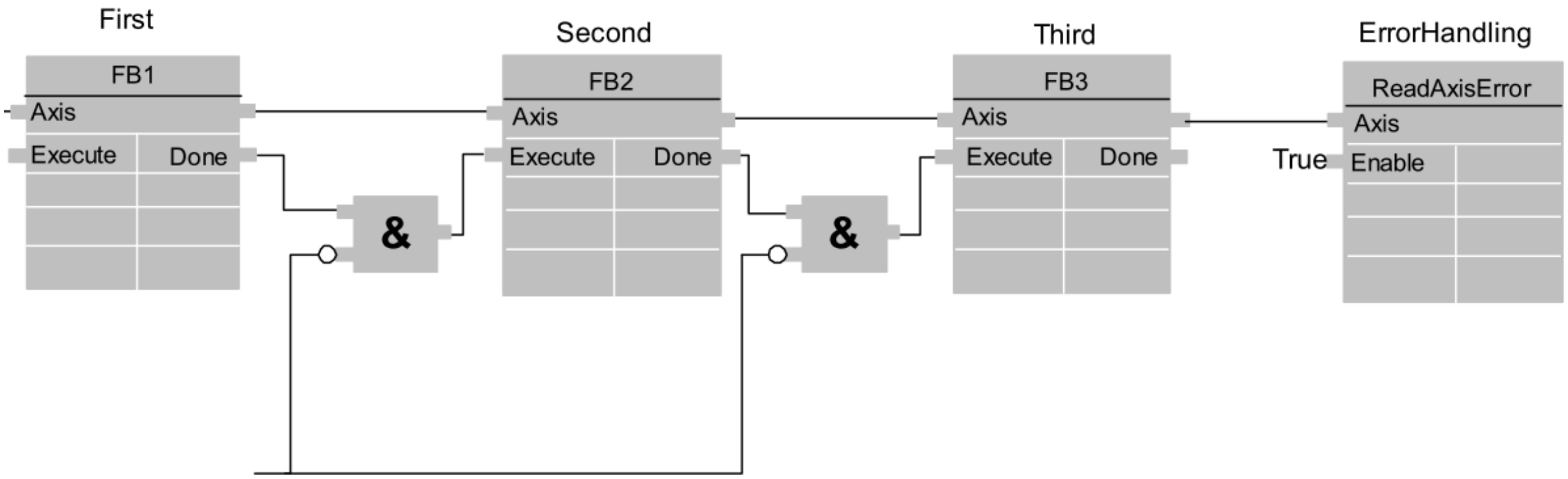
- *Done bit turns on*
  - *At least 1 scan*
  - *At command completion*
- *Done ≠ Position Complete*



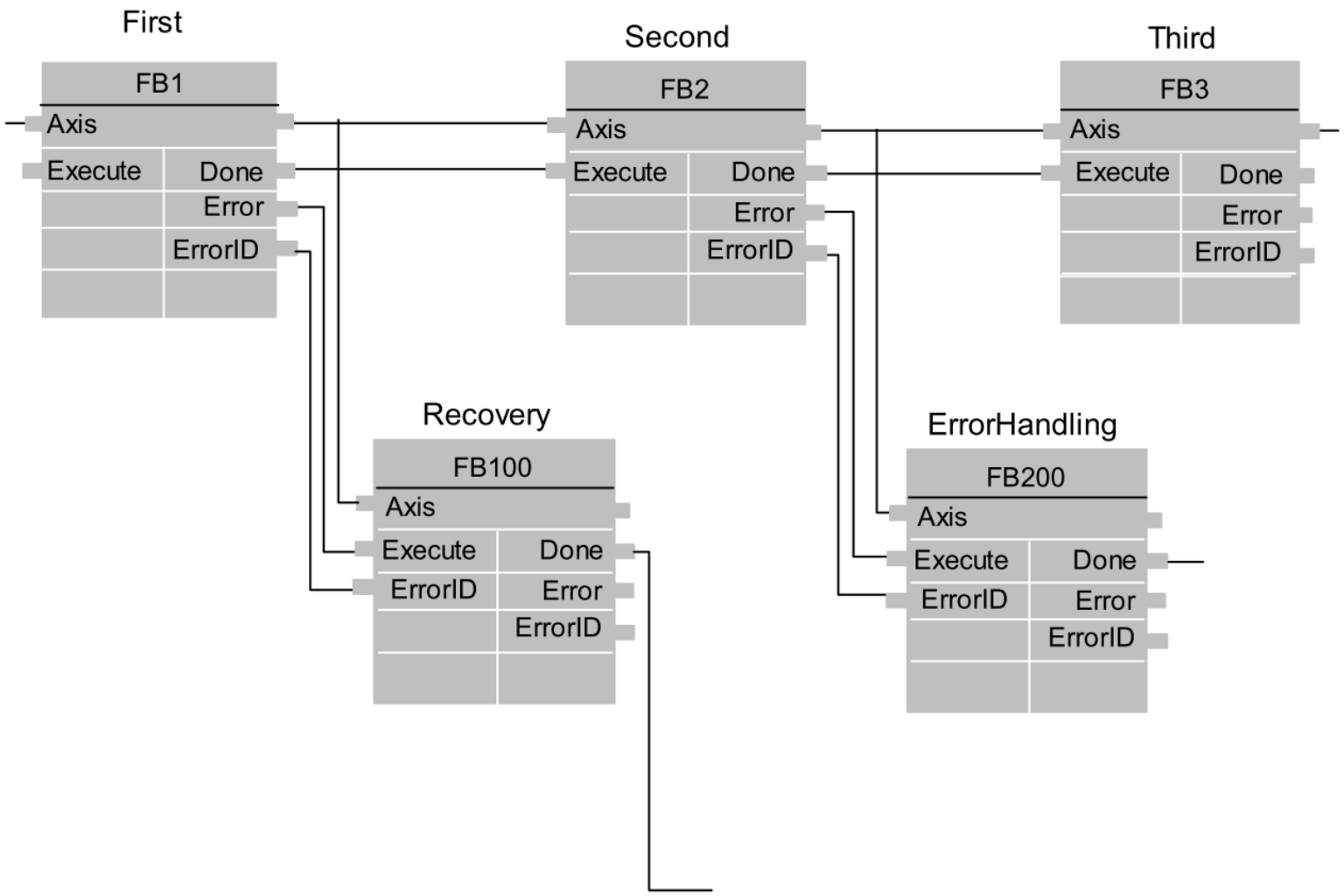




Function Blocks with centralized error handling



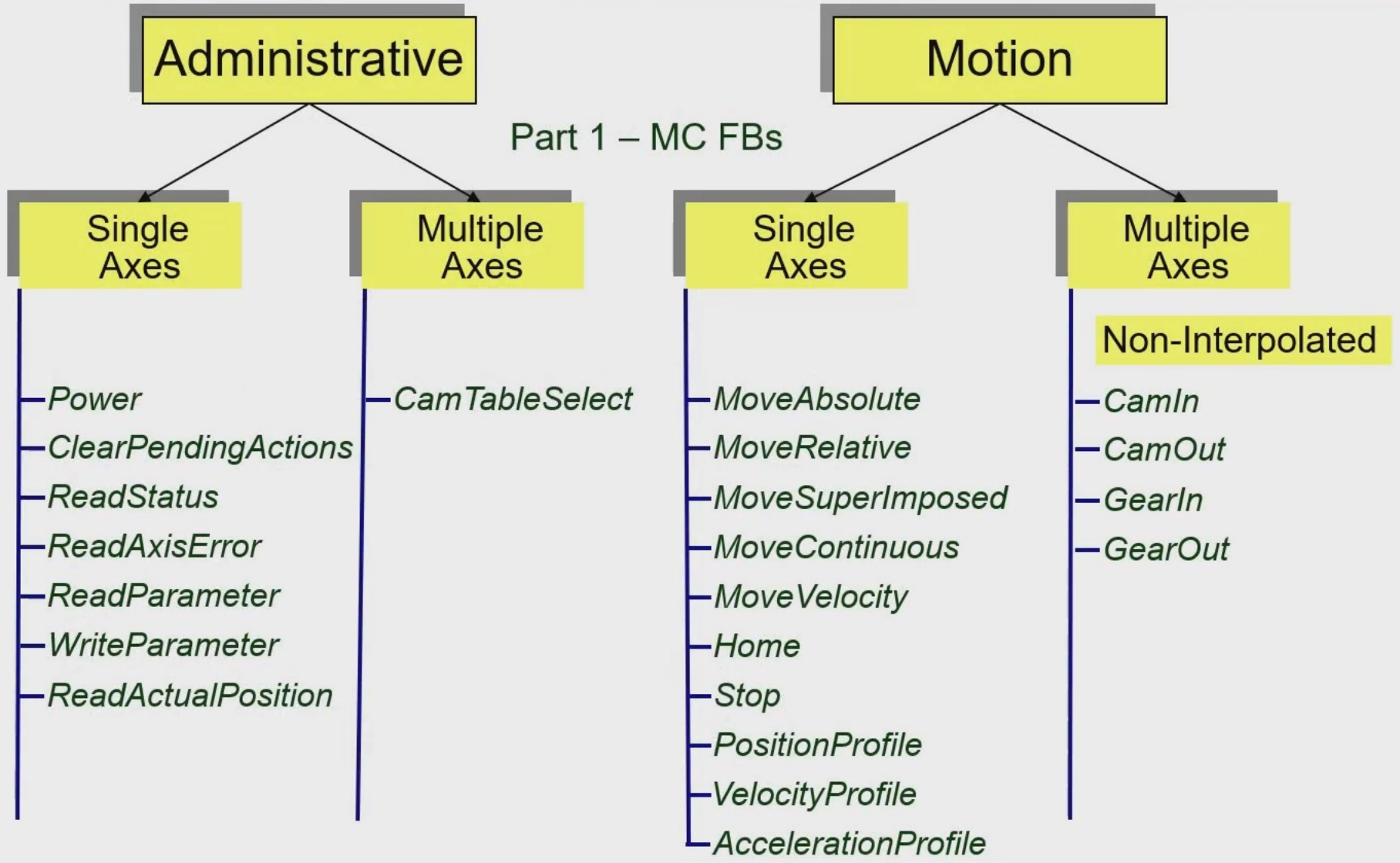
Function blocks with decentralized error handling



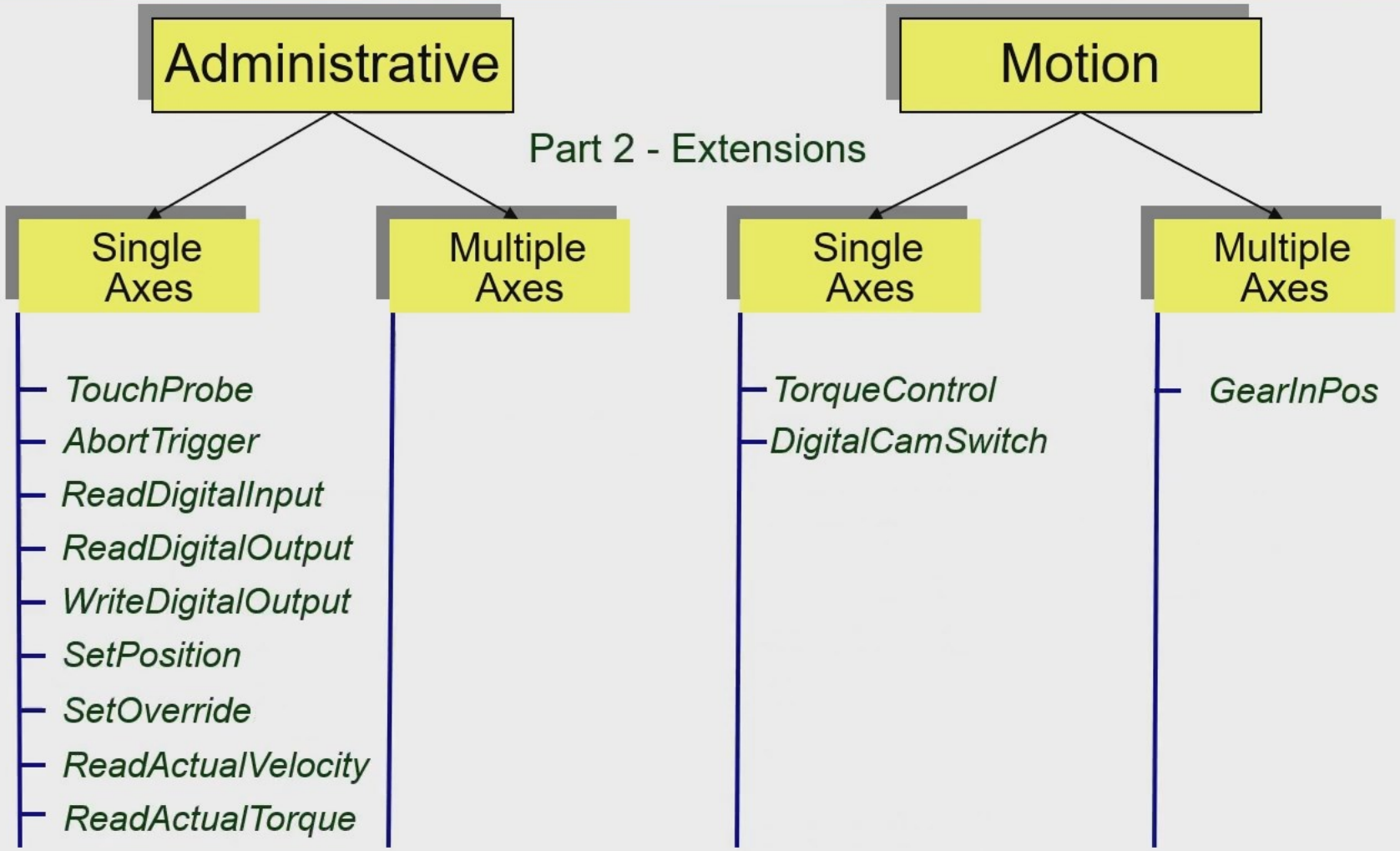
PLCopen Motion Control

- Part 1 – Function Blocks for Motion Control
- Part 2 – Extensions
- Part 3 – User Guidelines
- Part 4 – Coordinated Motion
- Part 5 – Homing procedures
- Part 6 – Fluid Power
- 30 companies certified with 67 products (check website for full list)

Part 1 – Basic FBs



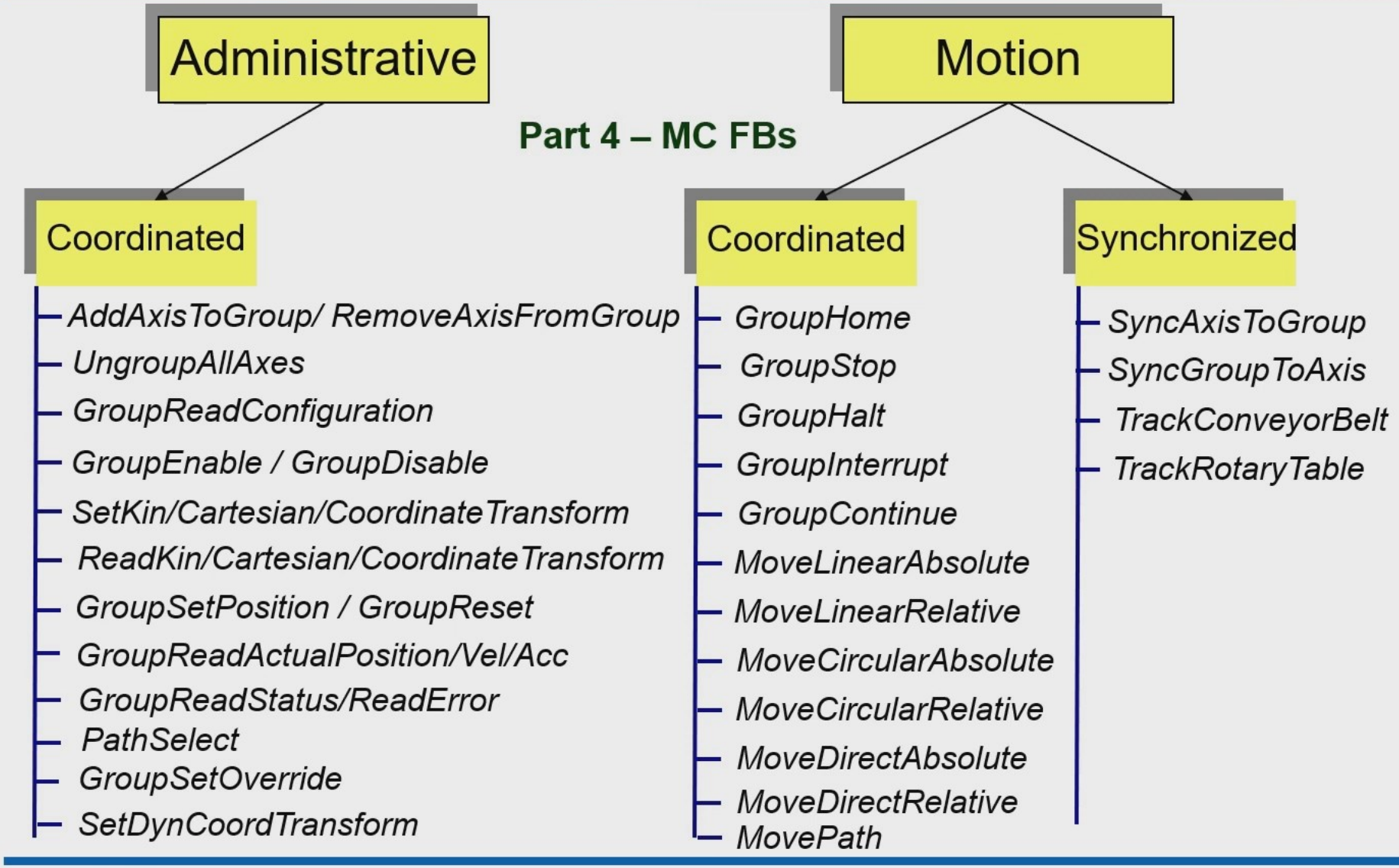
Part 2 - Extensions



Part 3 – User Guidelines

- Shows examples for ease-of-use
- Shows user-derived Function Blocks
- Shows higher level encapsulation (e.g. Winding)
- Stresses the creation of own FB libraries
- Uses FBD, LD, and ST
- 94 pages in total
- Not a training guideline

Part 4 – Coordinated Motion



Part 5 - Homing Procedures & FBs

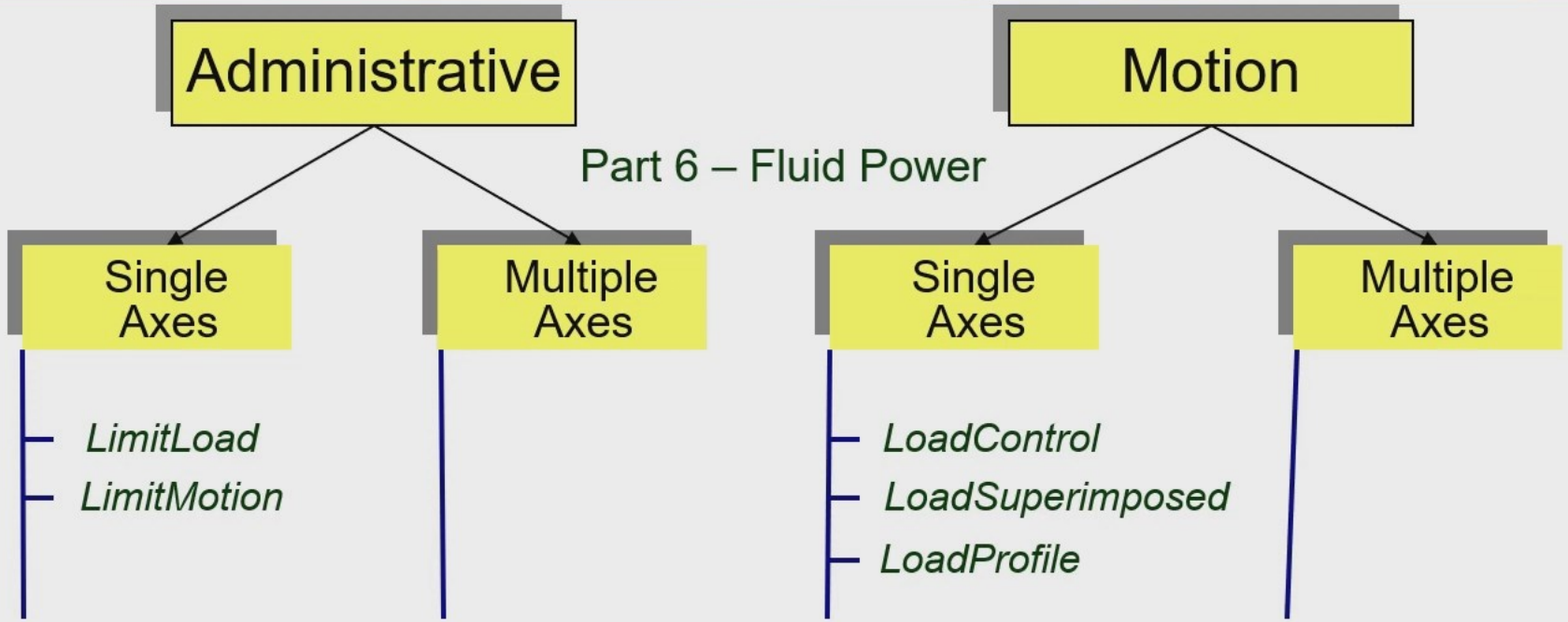
HomeAbsoluteSwitch
HomeLimitSwitch
HomeBlock
HomeReferencePulse
HomeReferencePulseSet
HomeDistanceCoded
HomeDirect
HomeAbsolute

Homing Function Blocks:
MC_StepAbsoluteSwitch,
MC_StepLimitSwitch, MC_StepBlock,
MC_StepReferencePulse,
MC_StepDistanceCoded

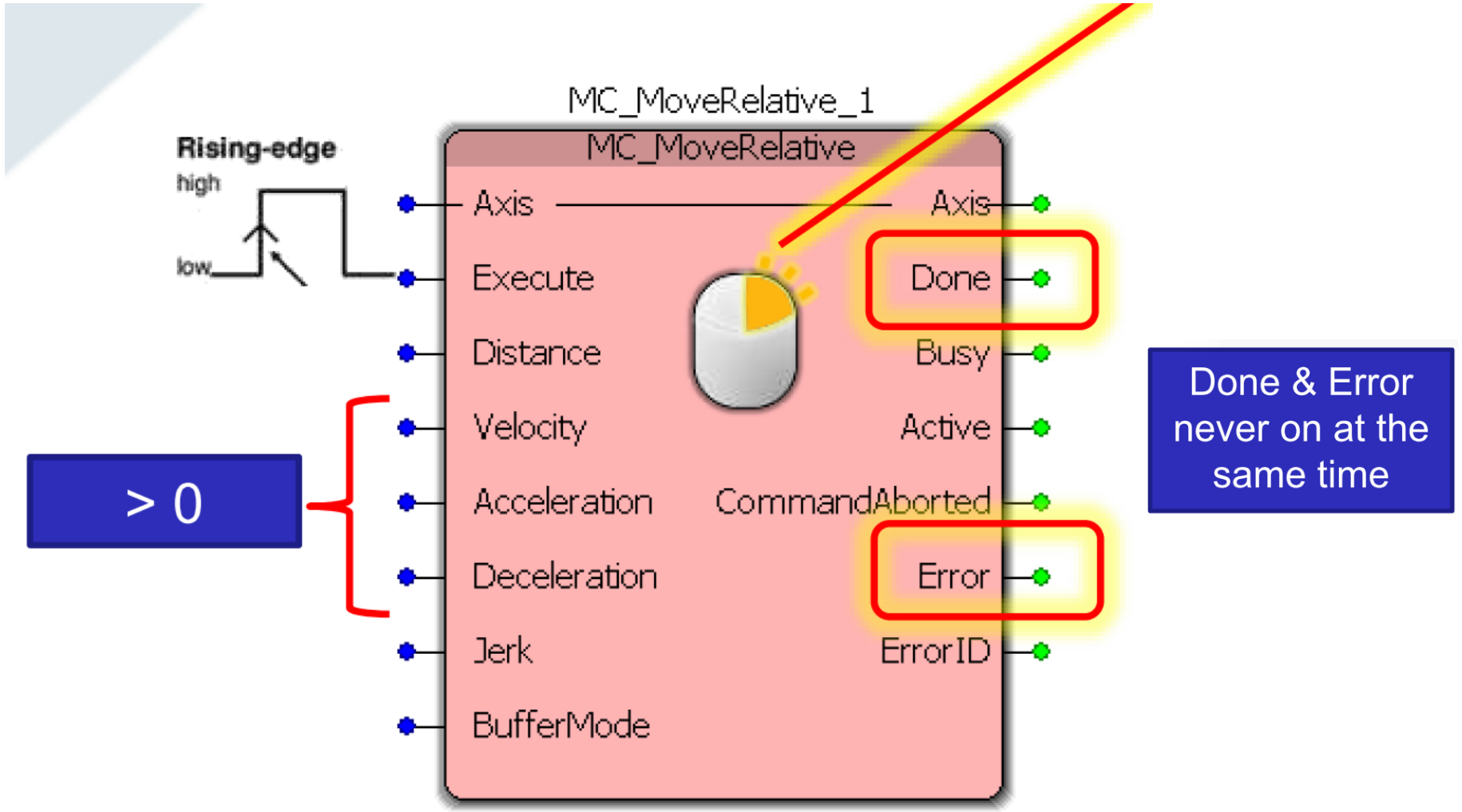
Finalizing homing:
MC_HomeDirect, MC_HomeAbsolute,
MC_FinishHoming

Homing on-the-fly:
MC_StepReferenceFlyingSwitch,
MC_StepReferenceFlyingRefPulse,
MC_AbortPassiveHoming

Part 6 – Extensions for Fluid Power

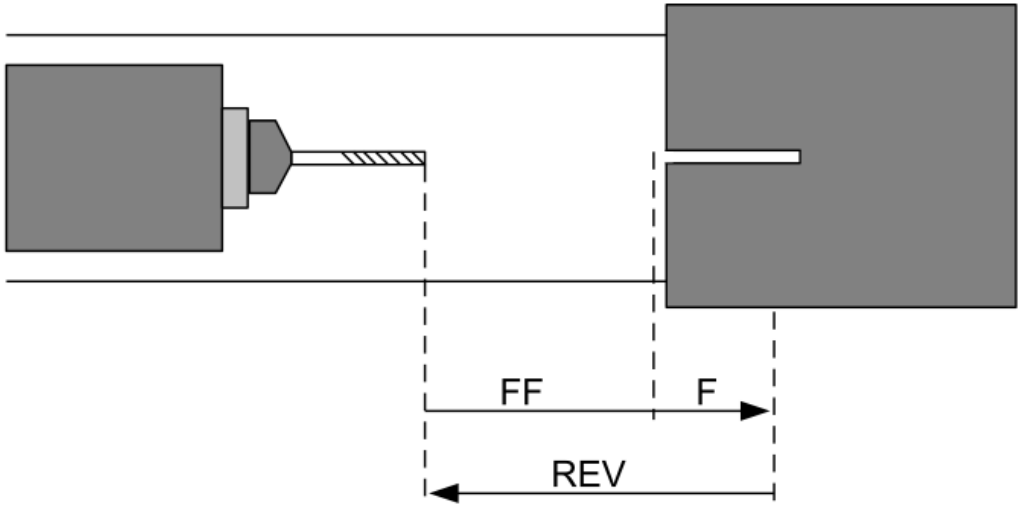


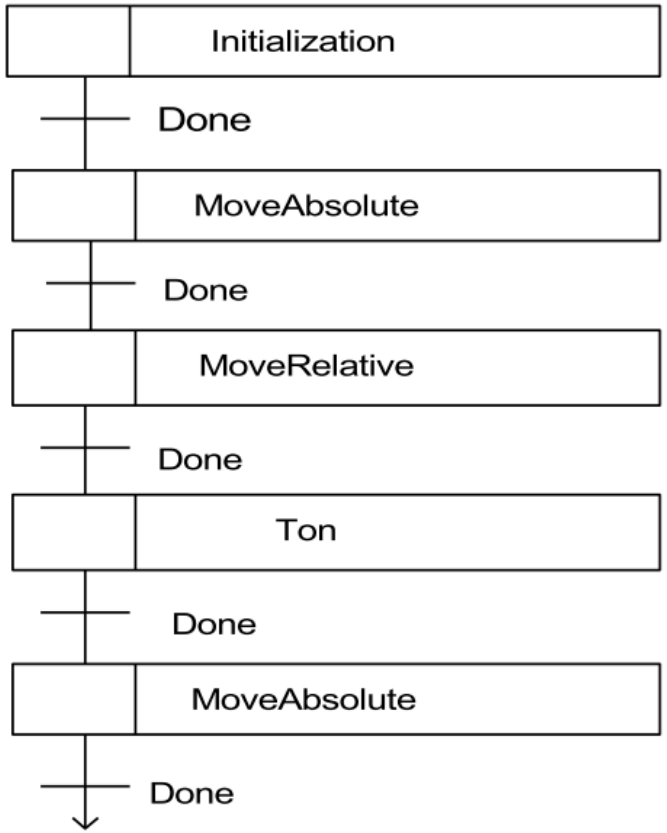
The function blocks for single axis motion control



MC_MoveRelative - moves the axis a specified distance relative to the actual position at the time of the execution;

Buffer mode	Short description Important note: The meaning of each value may vary depending on the FB(s) involved. For this reason, please also refer to the individual parameter descriptions!
Aborting	This is the Default mode. The FB aborts an ongoing motion and the command affects the axis immediately.
Buffered	The FB affects the axis as soon as the previous movement is complete. The axis will stop between the movements.
BlendingLow	The FB controls the axis after the previous FB has finished, but the axis will not stop between the movements. The velocity is blended with the lowest velocity of both commands.
BlendingPrevious	The FB controls the axis after the previous FB has finished (equivalent to buffered), but the axis wi exll not stop between the movements. Blending with the velocity of the previous move.
BlendingNext	The FB controls the axis after the previous FB has finished, but the axis will not stop between the movements. Blending with velocity of this (next) function.
BlendingHigh	The FB controls the axis after the previous FB has finished (equivalent to buffered), but the axis will not stop between the movements. Blending with highest velocity of the previous and this (next) function.





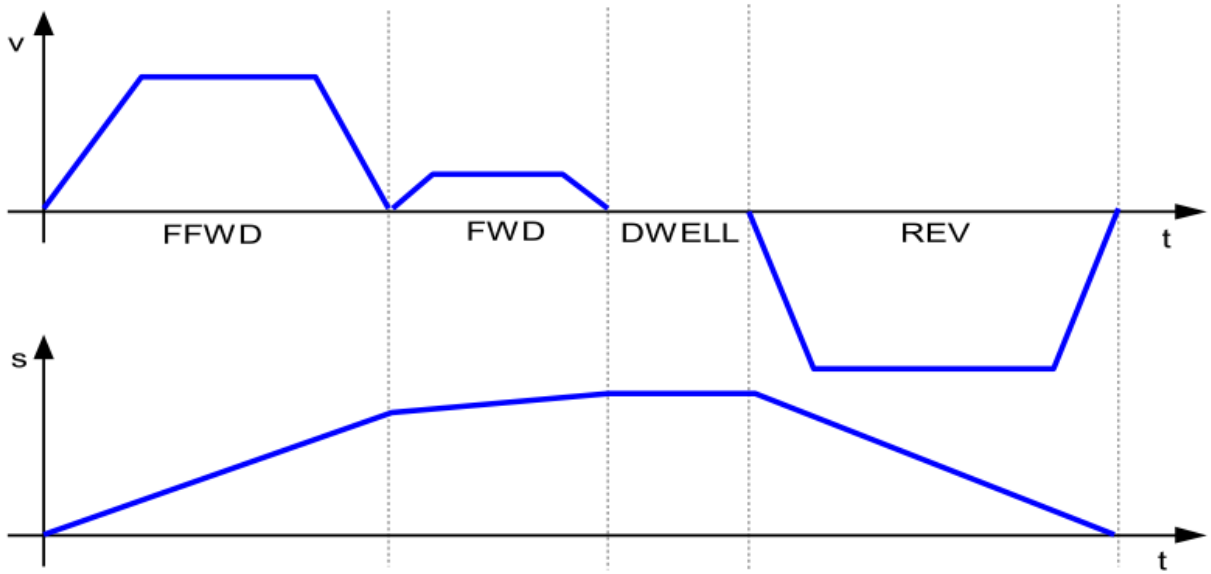
Step 1: Initialization, for instance at power up;

Step 2: Move forward to drilling position and start driller turning: in this way it will be fully operational before the position is reached; then check if both actions are completed;

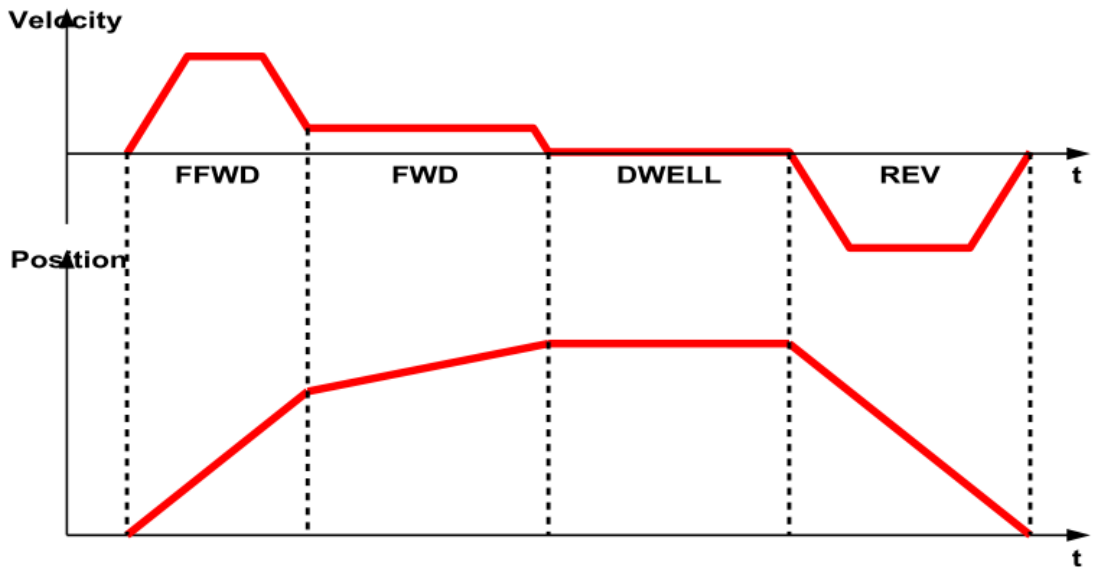
Step 3: Drill the hole;

Step 4: After drilling the hole we have to wait for the step-chain sequence to finish dwelling the hole free of any stuff which might have stuck in the hole;

Step 5: Move driller back to starting position and shut the spindle off. Combining the finishing of moving backwards and stopping the spindle we signal the step-chain to start over.



Timing diagram for drilling – aborting mode



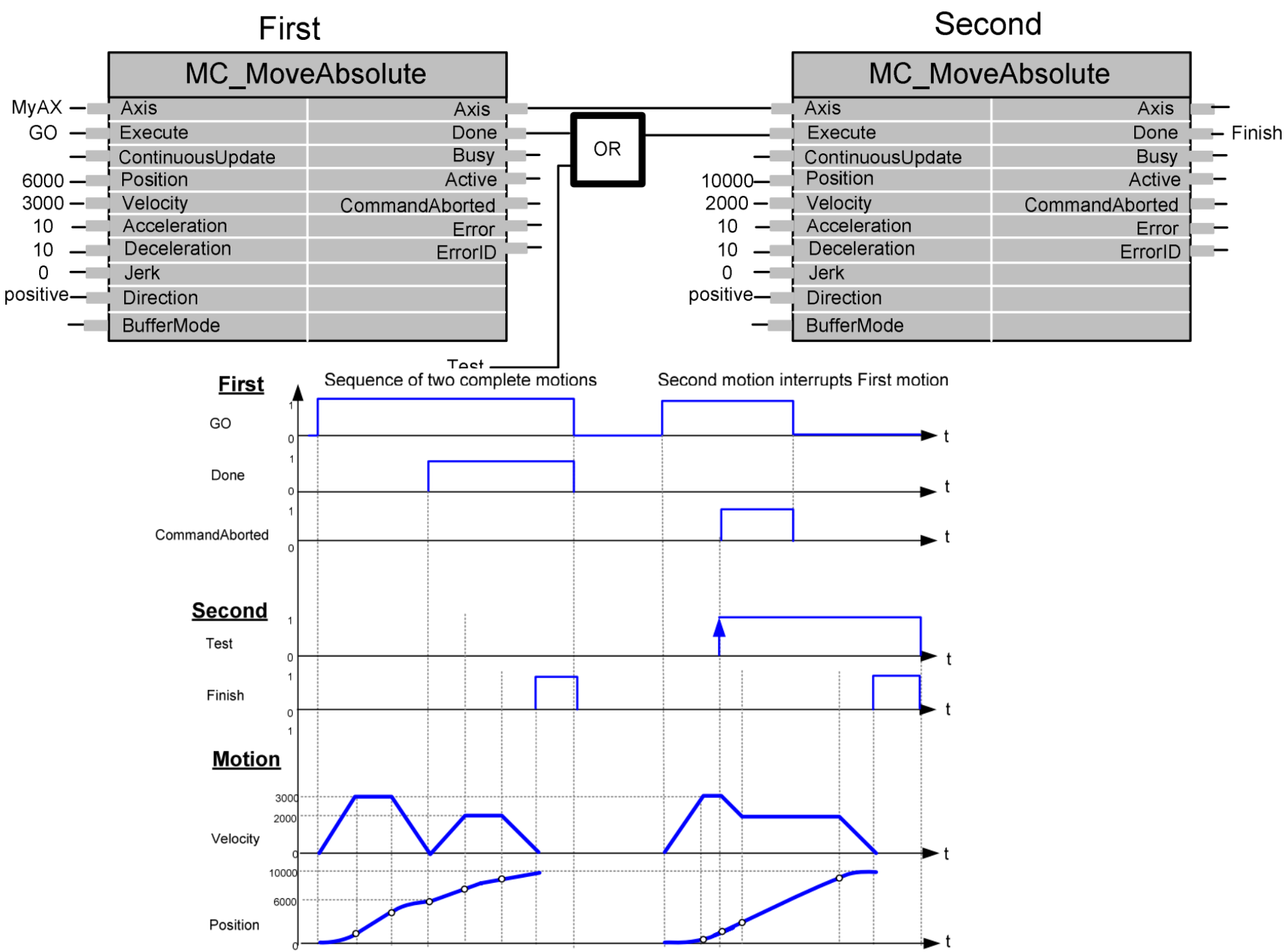
Timing diagram for drilling – blending mode

- **MC_MoveAbsolute** - this function block commands a controlled motion at a specified absolute position.
- **MC_MoveRelative** - moves the axis a specified distance relative to the actual position at the time of the execution;
- **MC_MoveAdditive** - for a specified relative distance additional to the original commanded position in the discrete motion state. In state Continuous Motion the specified relative distance is added to the actual position at the time of the execution;
- **MC_MoveSuperimposed** - for a specified relative distance additional to an existing motion. The existing Motion is not interrupted, but is superimposed by the additional motion;
- **MC_HaltSuperimposed** - commands a halt to all superimposed motions of the axis. The underlying motion is not interrupted;
- **MC_MoveVelocity** - for a never ending controlled motion at a specified velocity;
- **MC_MoveContinuousAbsolute** & **MC_MoveContinuousRelative** - commands a controlled motion to a specified absolute or relative position ending with the specified velocity;

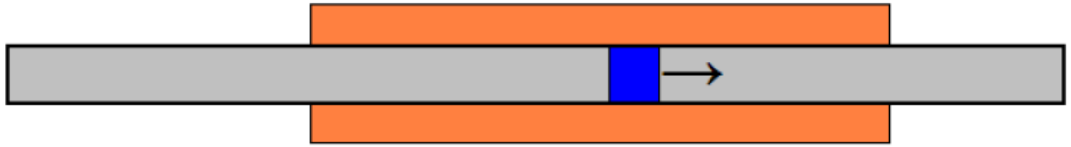
- **MC_TorqueControl -** continuously exerts a torque or force of the specified magnitude, approached using a defined ramp, and sets the ‘InTorque’ output if the torque level is reached;
- **MC_SetPosition** - shifts the coordinate system of an axis by manipulating both the set-point position as well as the actual position of an axis with the same value without any movement caused;
- **MC_SetOverride -** sets the values of override for the whole axis and all functions that are working on that axis;
- **MC_TouchProbe** - records an axis position at a trigger event;
- **MC_AbortTrigger** - is used to abort function blocks which are connected to trigger events (e.g. MC_TouchProbe);
- **MC_DigitalCamSwitch** - provides the analogy to switches on a motor shaft: it commands a group of discrete output bits to switch in analogy to a set of mechanical cam controlled switches connected to an axis. Forward and backward movements are allowed;
- **MC_Home** - commands the axis to perform the «search home» sequence. The details of this sequence are manufacturer dependent and can be set by axis parameters, as well as the function blocks as defined in Part 5 – Homing Procedures;

- **MC_Stop** - commands a controlled motion stop and transfers the axis to the state ‘Stopping’. It aborts any ongoing function block execution. With the ‘Done’ output set, the state is transferred to the ‘StandStill”. While the axis is in state ‘Stopping’ no other FB can perform any motion on the same axis;
- **MC_Halt** - commands a controlled motion stop. It aborts any ongoing function block execution. The axis is moved to the state ‘DiscreteMotion’ until the velocity is zero. With the ‘Done’ output set, the state is transferred to ‘StandStill’;
- **MC_Power** - switches the power stage on or off;
- **MC_ReadStatus** - returns in detail the status of the state diagram of the selected axis;
- **MC_ReadMotionState** - returns in detail the status of the axis with respect to the motion currently in progress;
- **MC_ReadAxisInfo** - reads information like modes, inputs directly related to the axis, and certain axis status information;
- **MC_ReadAxisError** - Indicates errors not relating to the function blocks;

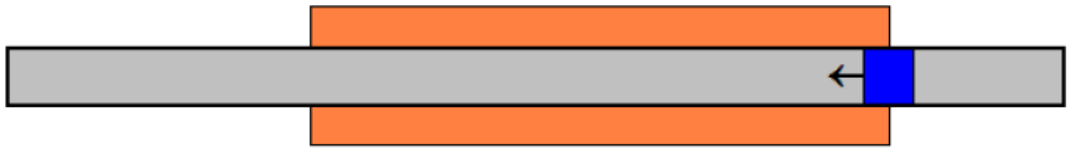
- **MC_Reset** - makes the transition from the state ‘ErrorStop’ to ‘StandStill’ by resetting all internal axis-related errors and clearing pending commands;
- **MC_ReadParameter** & **MC_ReadBoolParameter** - Returns the value of a vendor specific parameter;
- **MC_WriteParameter** & **MC_WriteBoolParameter** - modifies the value of a vendor specific parameter;
- **MC_ReadActualPosition** - returns the actual position;
- **MC_ReadDigitalInput** - provides the value of the digital input as referenced by INPUT_REF;


First
MC_MoveAbsolute
MyAX
GO
6000
3000
10
10
0
positive
Axis
Execute
ContinuousUpdate
Position
Velocity
Acceleration
Deceleration
Jerk
Direction
BufferMode
Axis
Done
Busy
Active
CommandAborted
Error
ErrorID
OR
Test
Second
MC_MoveAbsolute
10000
2000
10
10
0
positive
Axis
Execute
ContinuousUpdate
Position
Velocity
Acceleration
Deceleration
Jerk
Direction
BufferMode
Axis
Done
Busy
Active
CommandAborted
Error
ErrorID
Finish
First
Sequence of two complete motions
Second motion interrupts First motion
GO
Done
CommandAborted
Second
Test
Finish
Motion
Velocity
Position
3000
2000
10000
6000
t

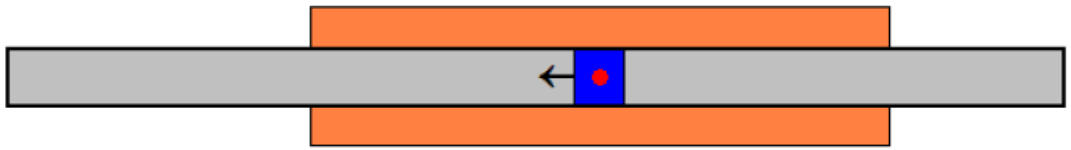
1. Move the laser with fast velocity over the position lrStartCutPos. The laser is off during this movement:



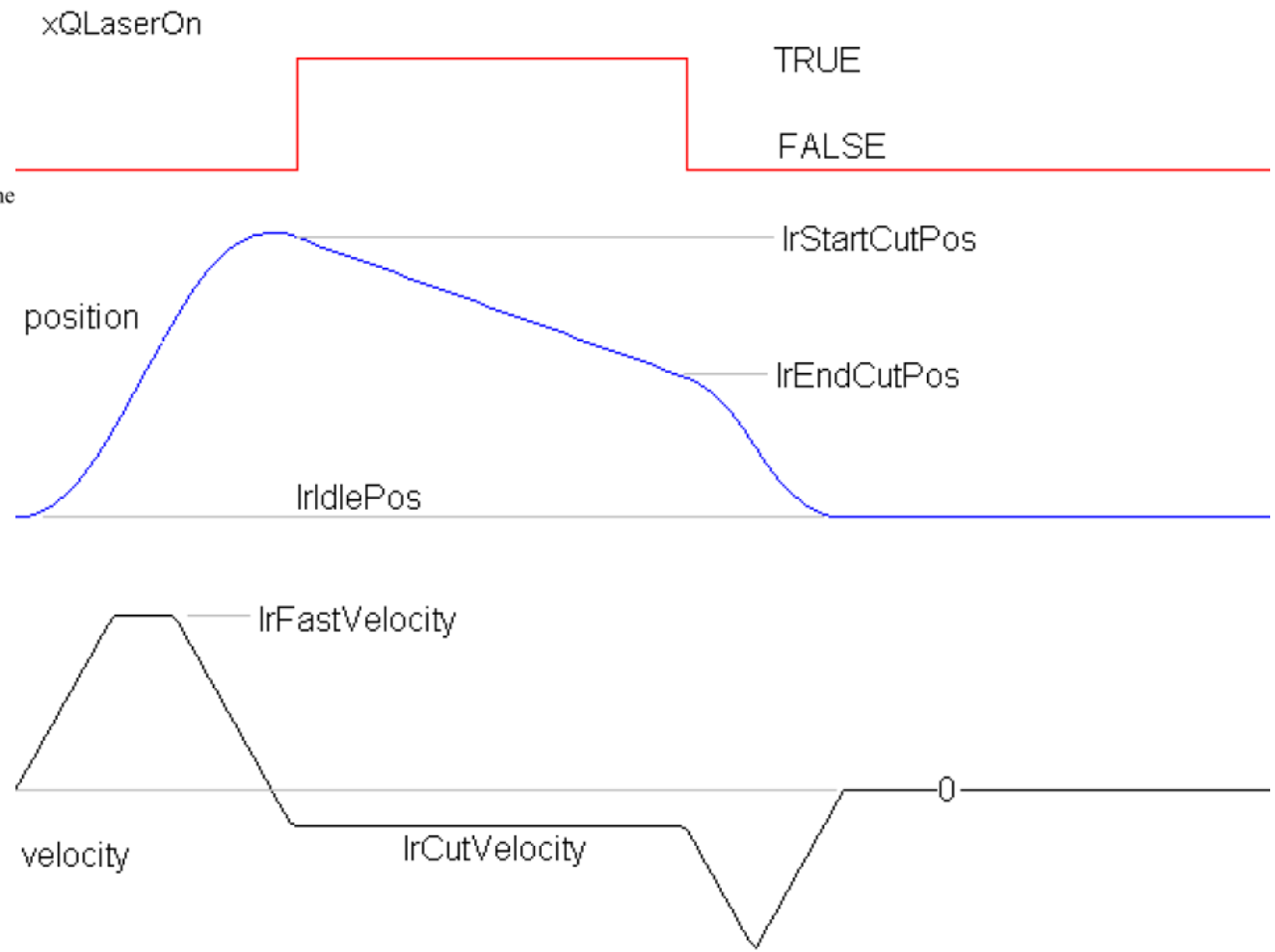
2. Turn back and make sure to have the speed lrCutVelocity when at lrStartCutPos. At this position, switch the ser on:

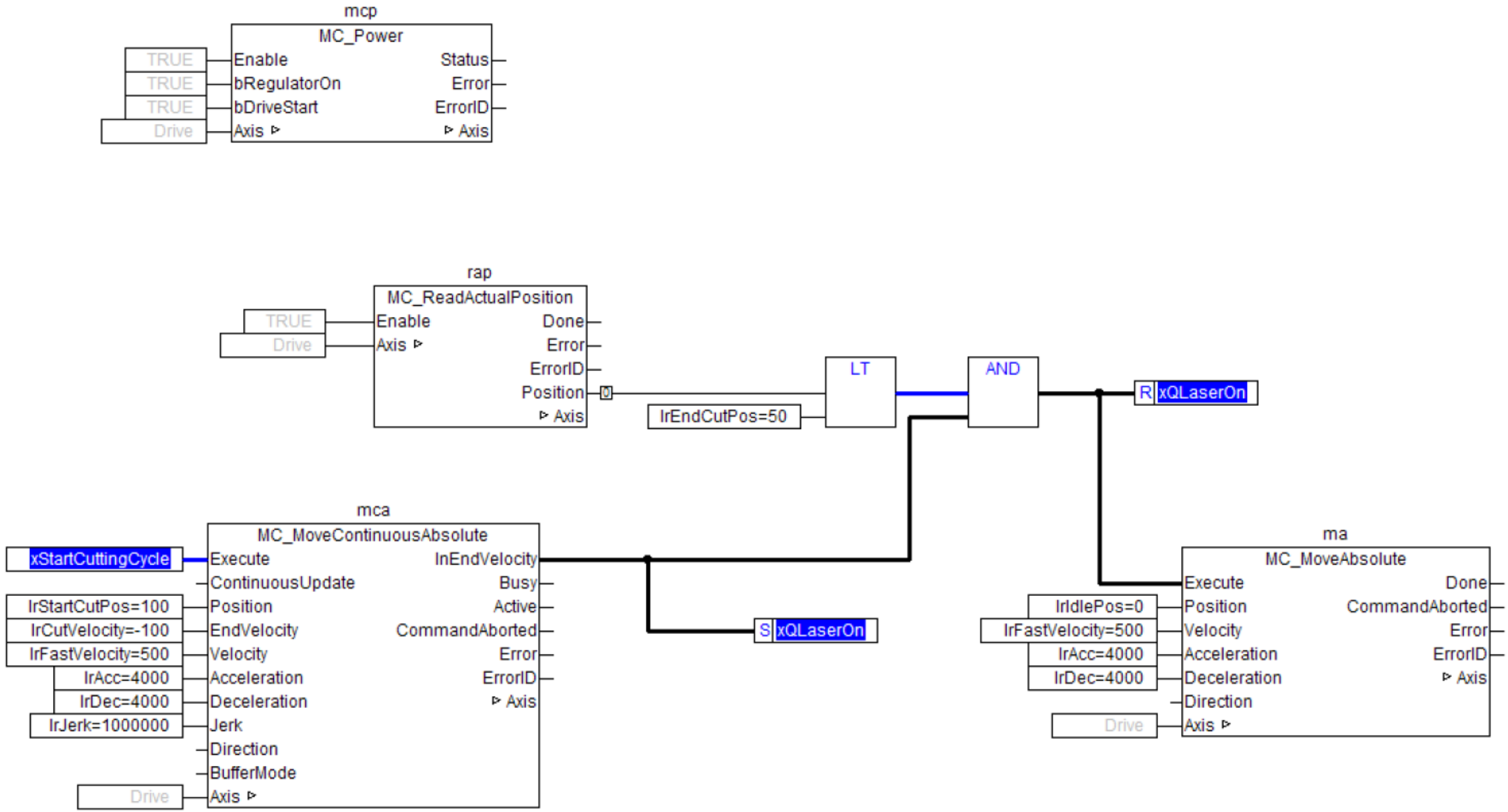


3. Travel over the work piece with this constant speed while the laser is on:

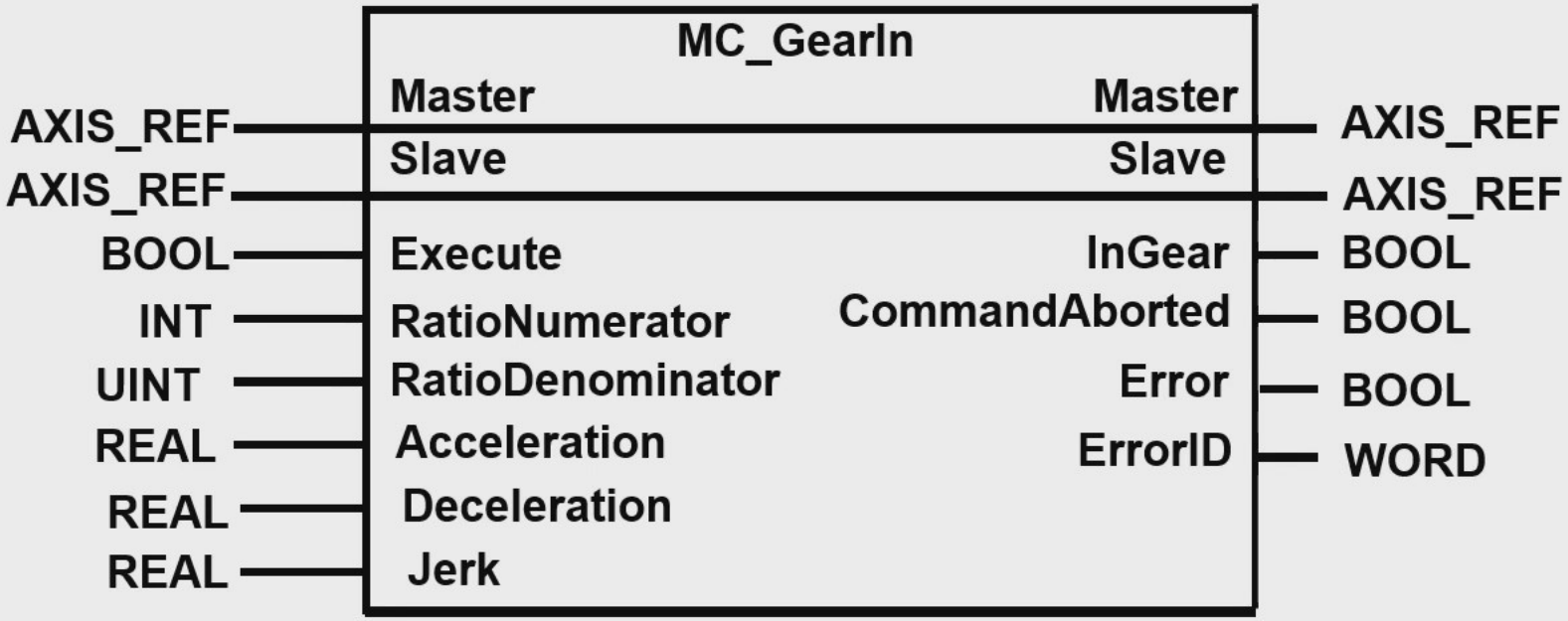


4. When reaching lrEndCutPos switch off the laser and move back to idle position with fast velocity:

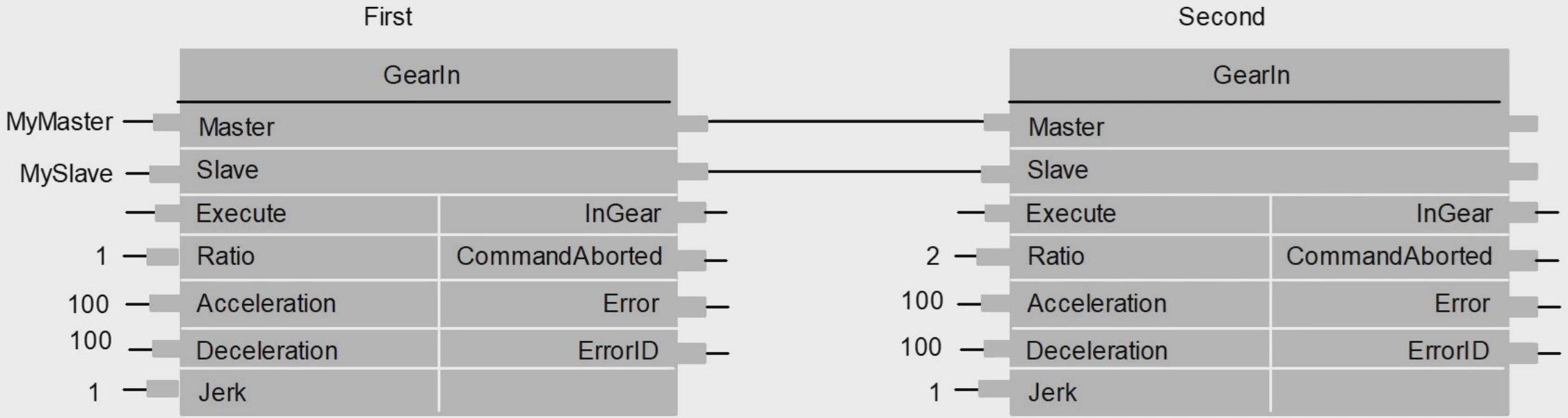



mcp
MC_Power
TRUE
TRUE
TRUE
Drive
Enable
bRegulatorOn
bDriveStart
Axis
Status
Error
ErrorID
Axis
rap
MC_ReadActualPosition
TRUE
Drive
Enable
Axis
Done
Error
ErrorID
Position
Axis
0
LT
lrEndCutPos=50
AND
R
xQLaserOn
mca
MC_MoveContinuousAbsolute
xStartCuttingCycle
lrStartCutPos=100
lrCutVelocity=-100
lrFastVelocity=500
lrAcc=4000
lrDec=4000
lrJerk=1000000
Drive
Execute
ContinuousUpdate
Position
EndVelocity
Velocity
Acceleration
Deceleration
Jerk
Direction
BufferMode
Axis
InEndVelocity
Busy
Active
CommandAborted
Error
ErrorID
Axis
S
xQLaserOn
ma
MC_MoveAbsolute
lrIdlePos=0
lrFastVelocity=500
lrAcc=4000
lrDec=4000
Drive
Execute
Position
Velocity
Acceleration
Deceleration
Direction
Axis
Done
CommandAborted
Error
ErrorID
Axis

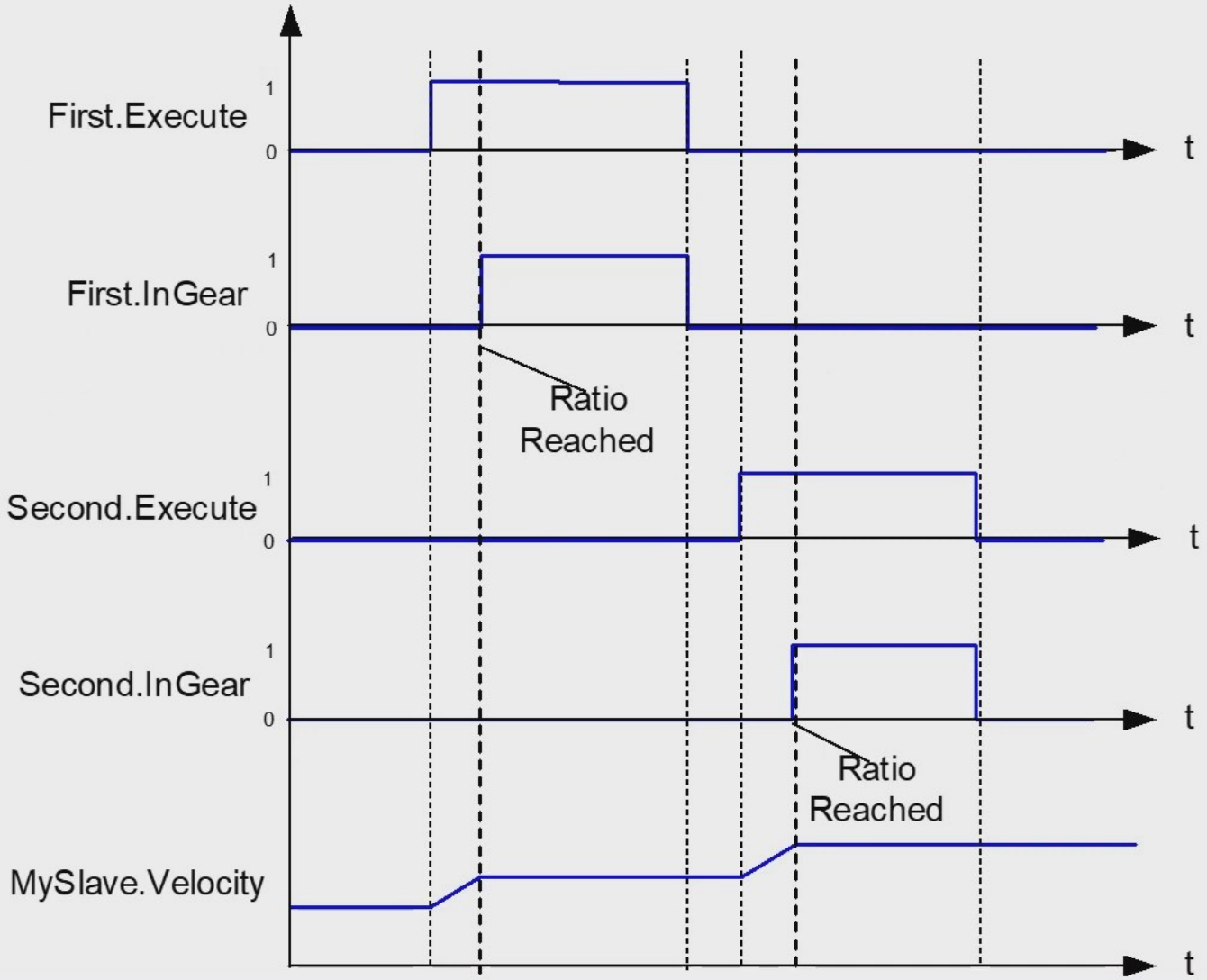
Example - GearIn

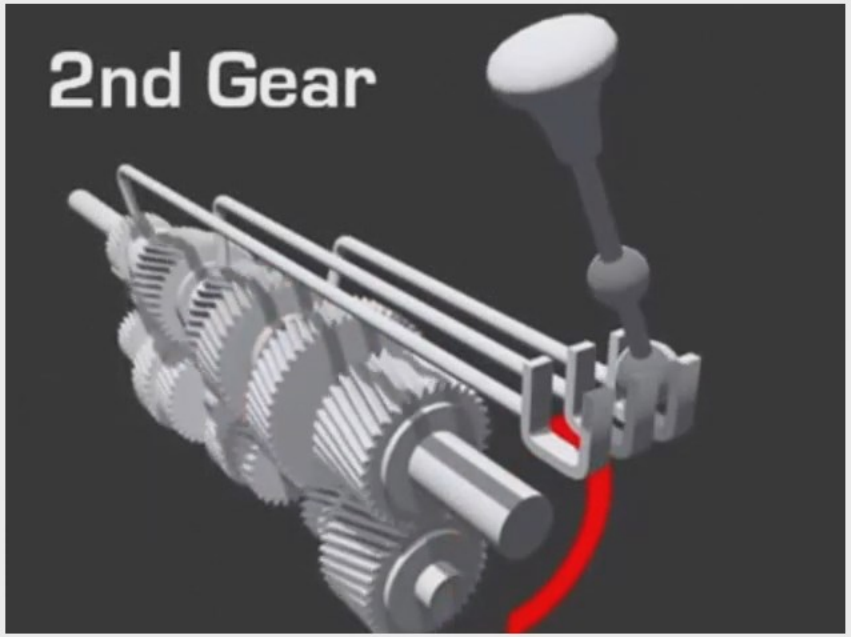


GearIn


First
GearIn
MyMaster
Master
MySlave
Slave
Execute
InGear
1
Ratio
CommandAborted
100
Acceleration
Error
100
Deceleration
ErrorID
1
Jerk
Second
GearIn
Master
Slave
Execute
InGear
2
Ratio
CommandAborted
100
Acceleration
Error
100
Deceleration
ErrorID
1
Jerk

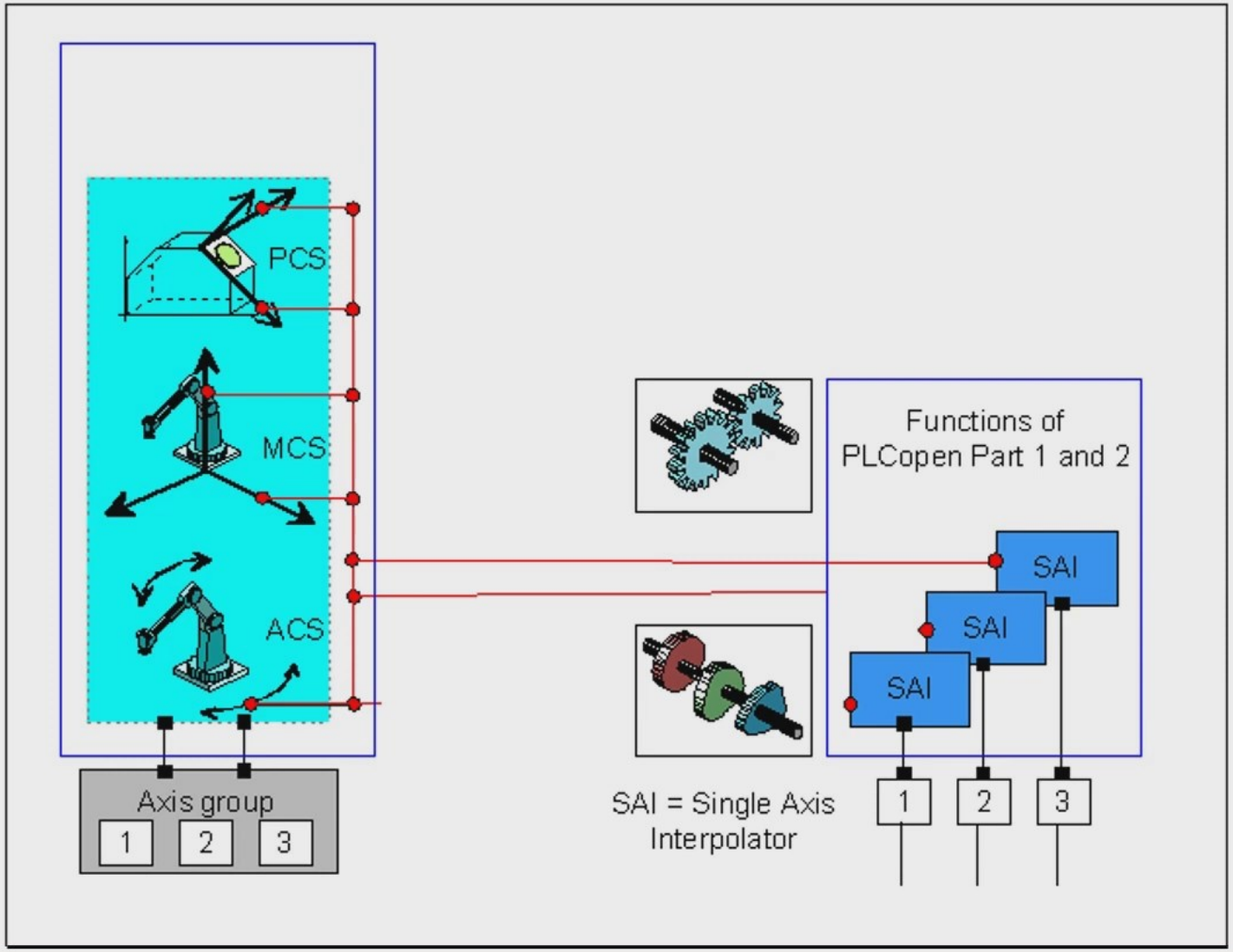
GearIn





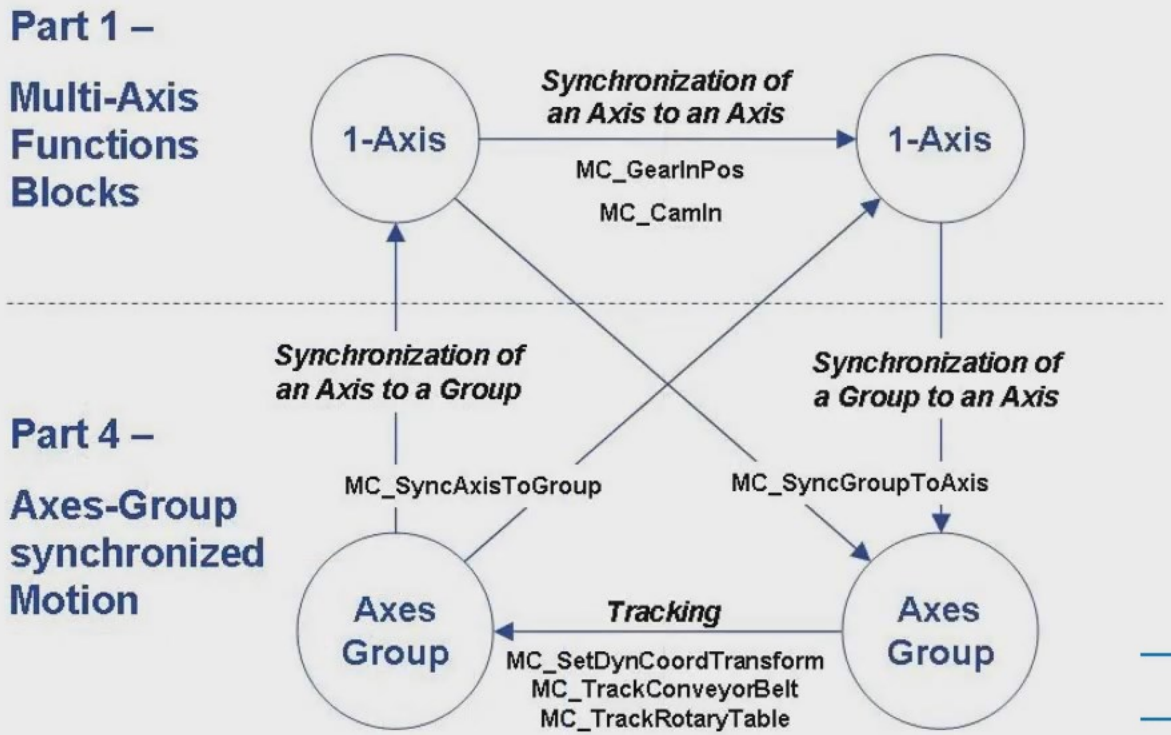
PLCopen®
motion control

Part 4 – Coordinated Motion

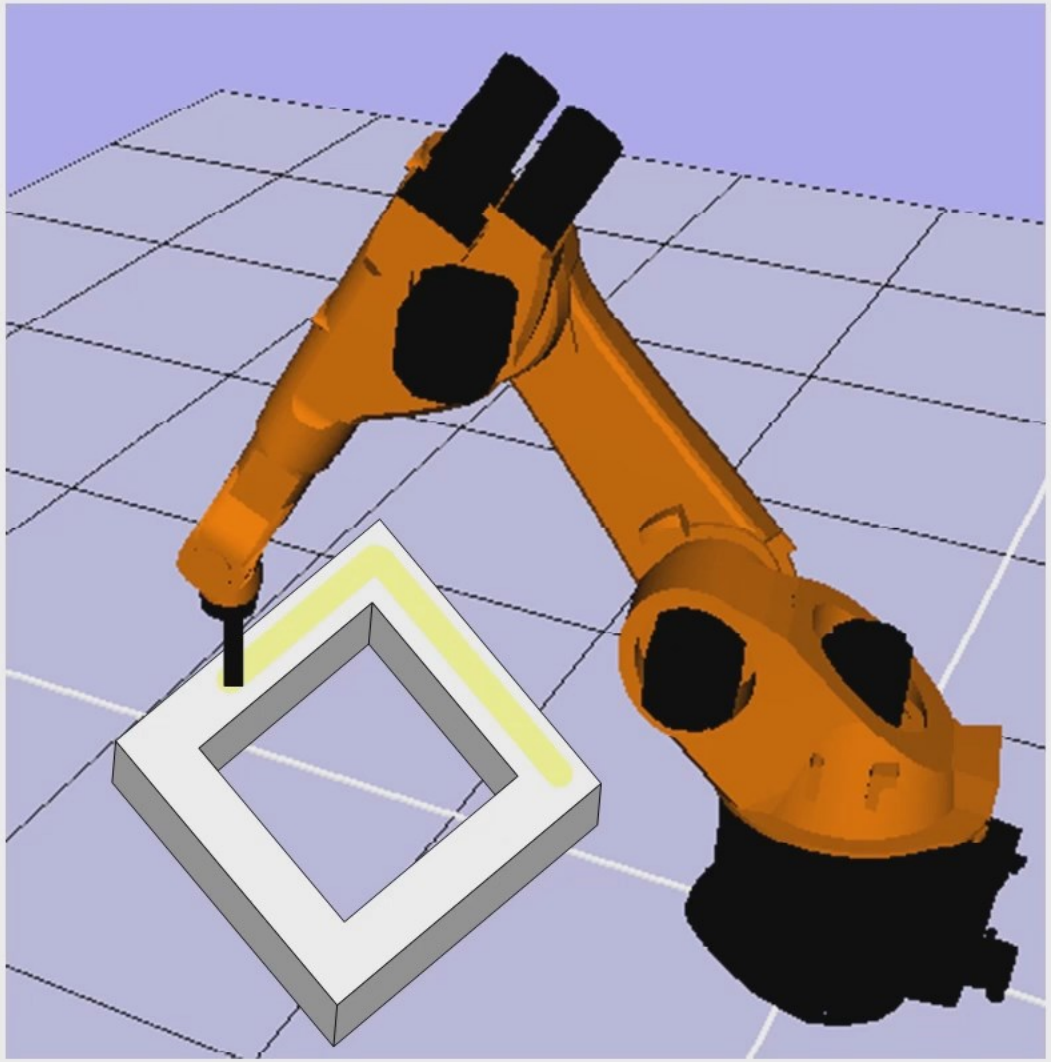


Synchronized Motion items

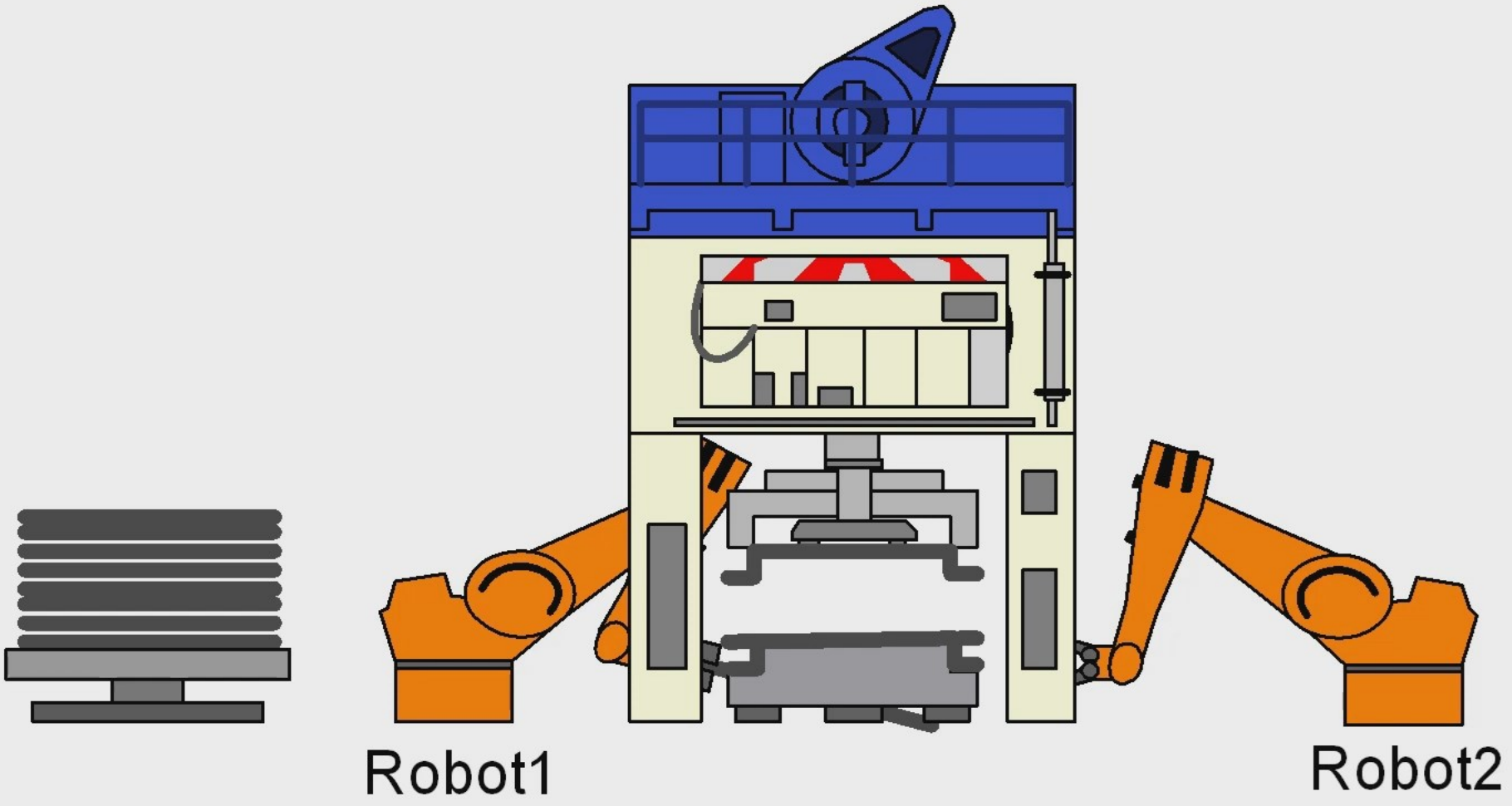
- SyncAxisToGroup
- SyncGroupToAxis
- TrackConveyorBelt
- TrackRotaryTable



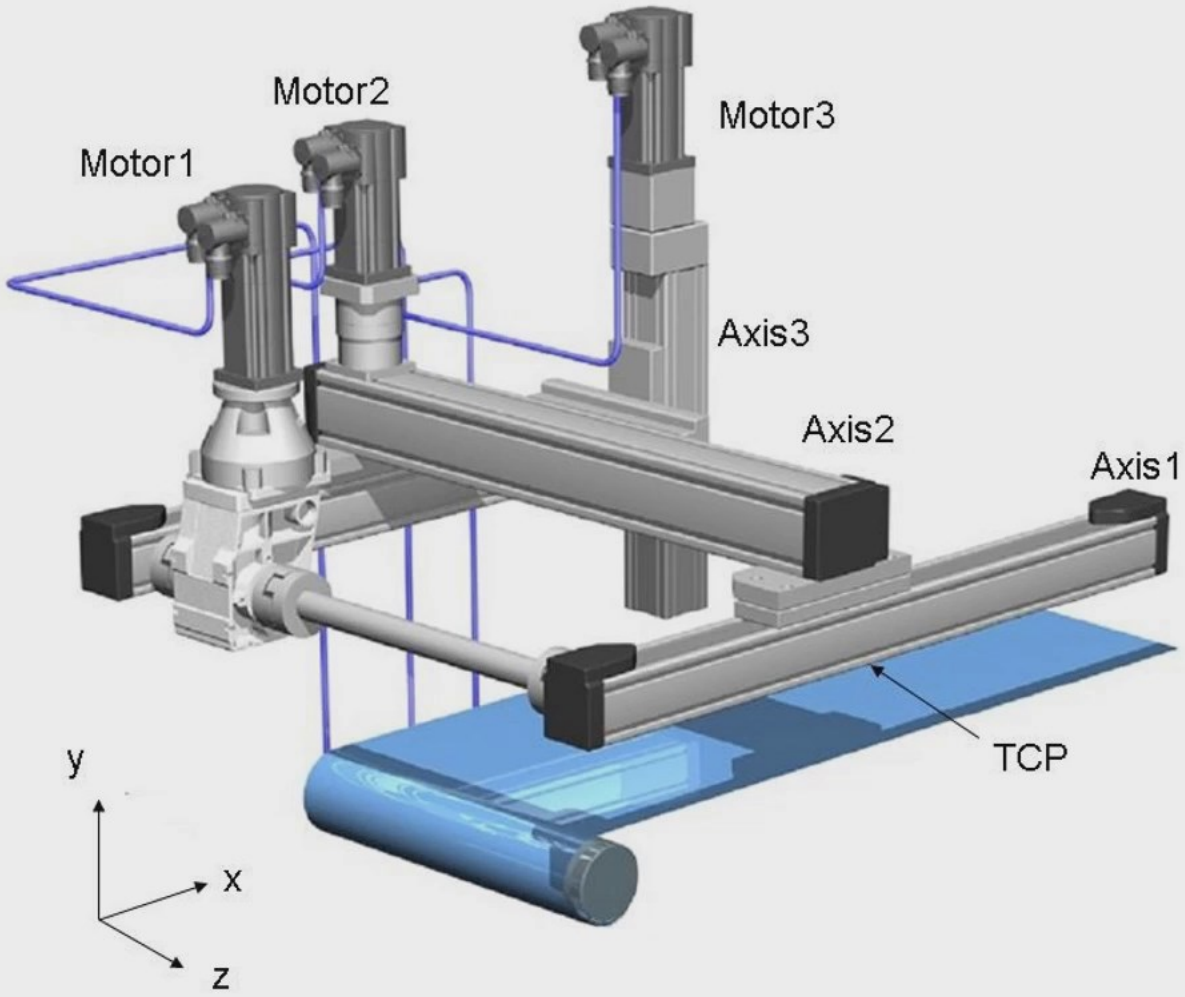
Synchronization of single axis to an axes group



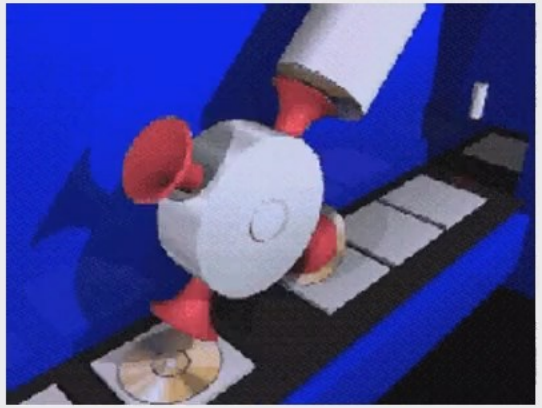
Synchronization of an axes group to a single axis

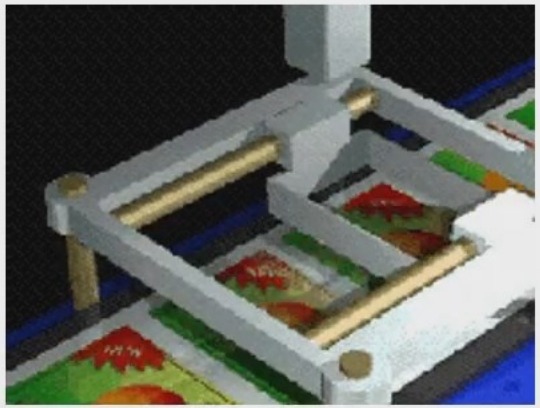


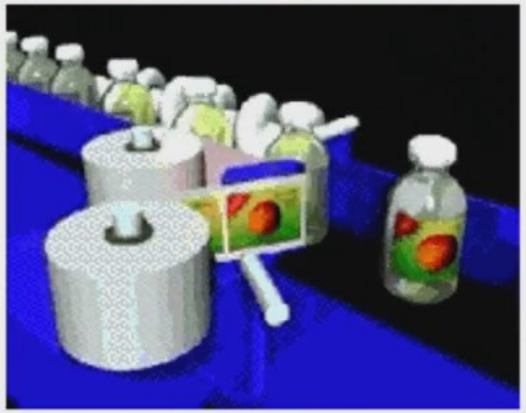
Example of tracking



Encapsulation

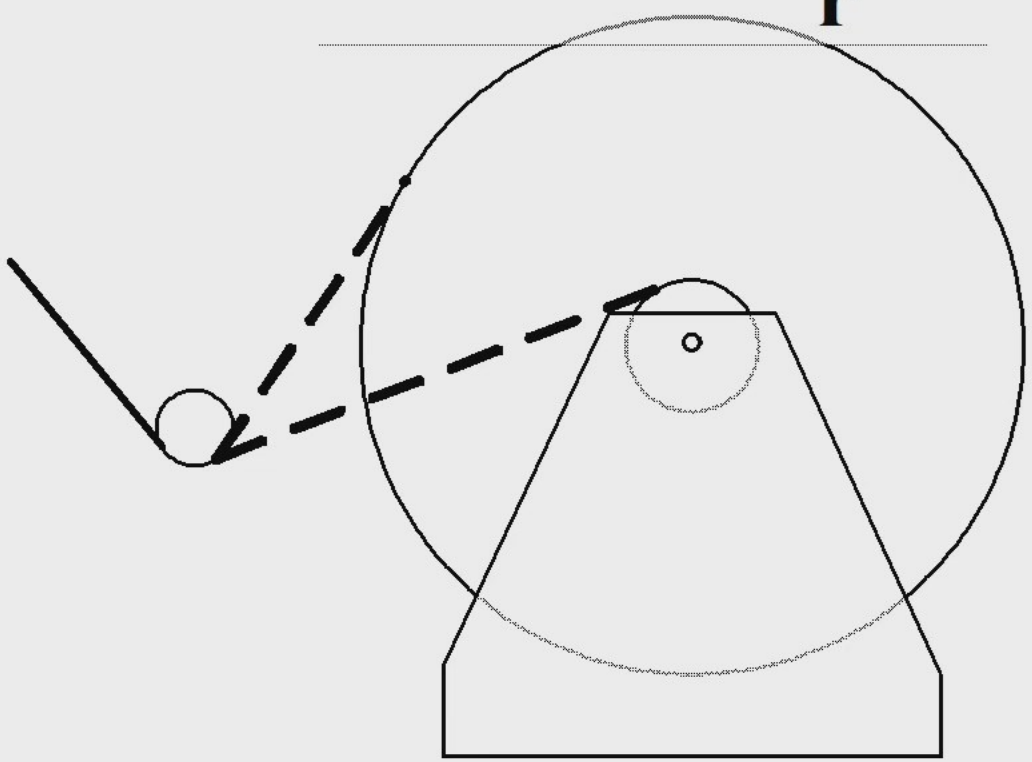




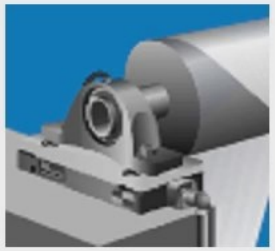


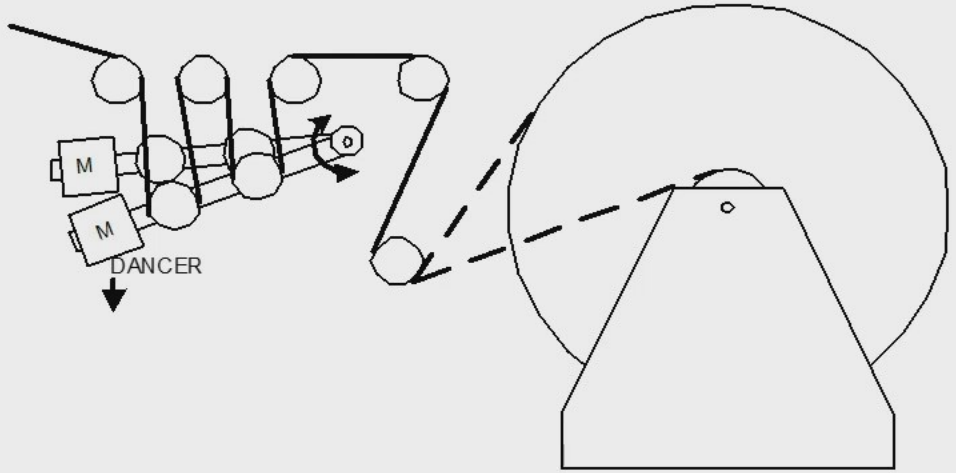
Winding / Unwinding

$$\text{tension} = \frac{\text{torque}}{r}$$

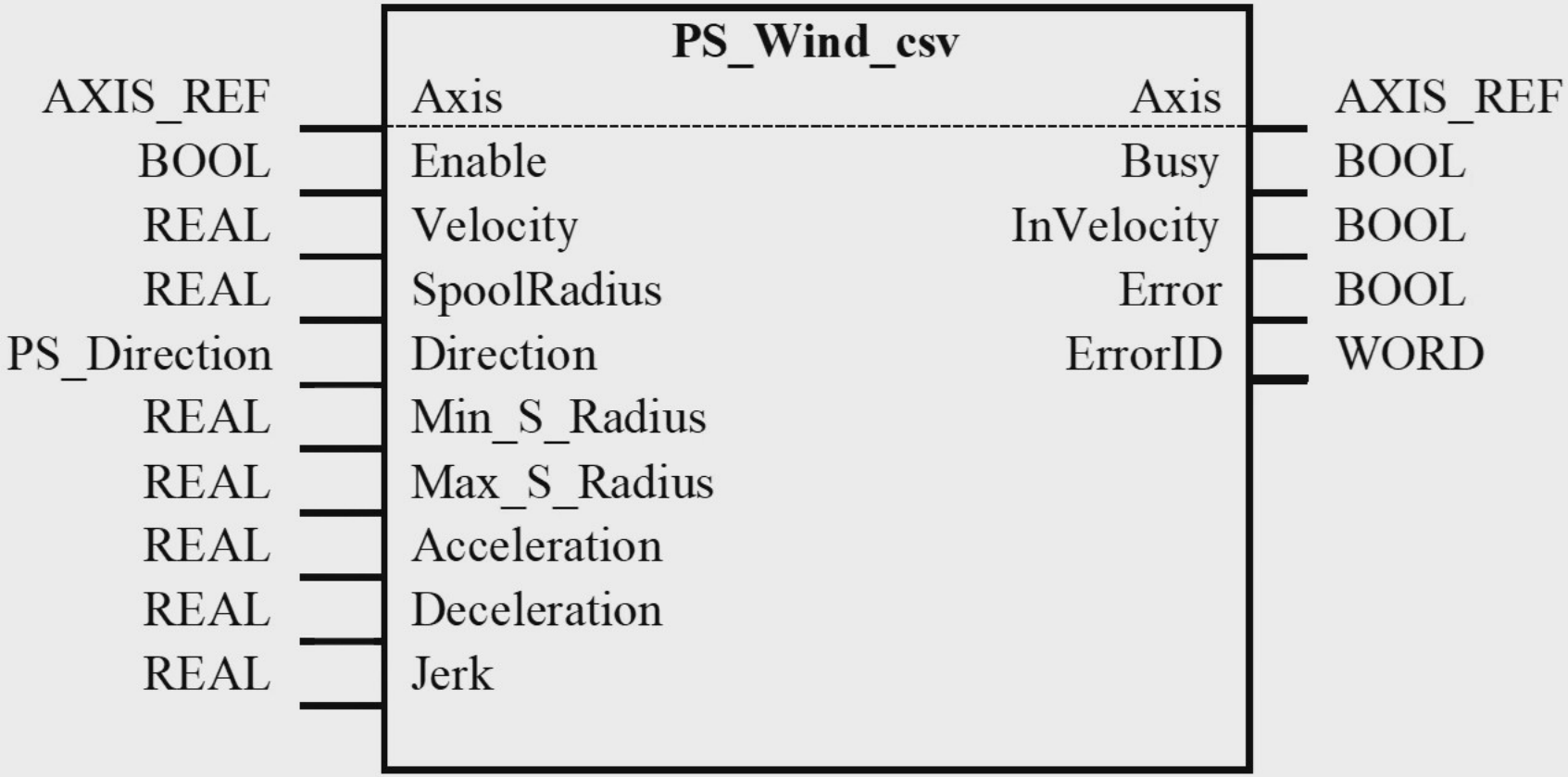


Dancer Control



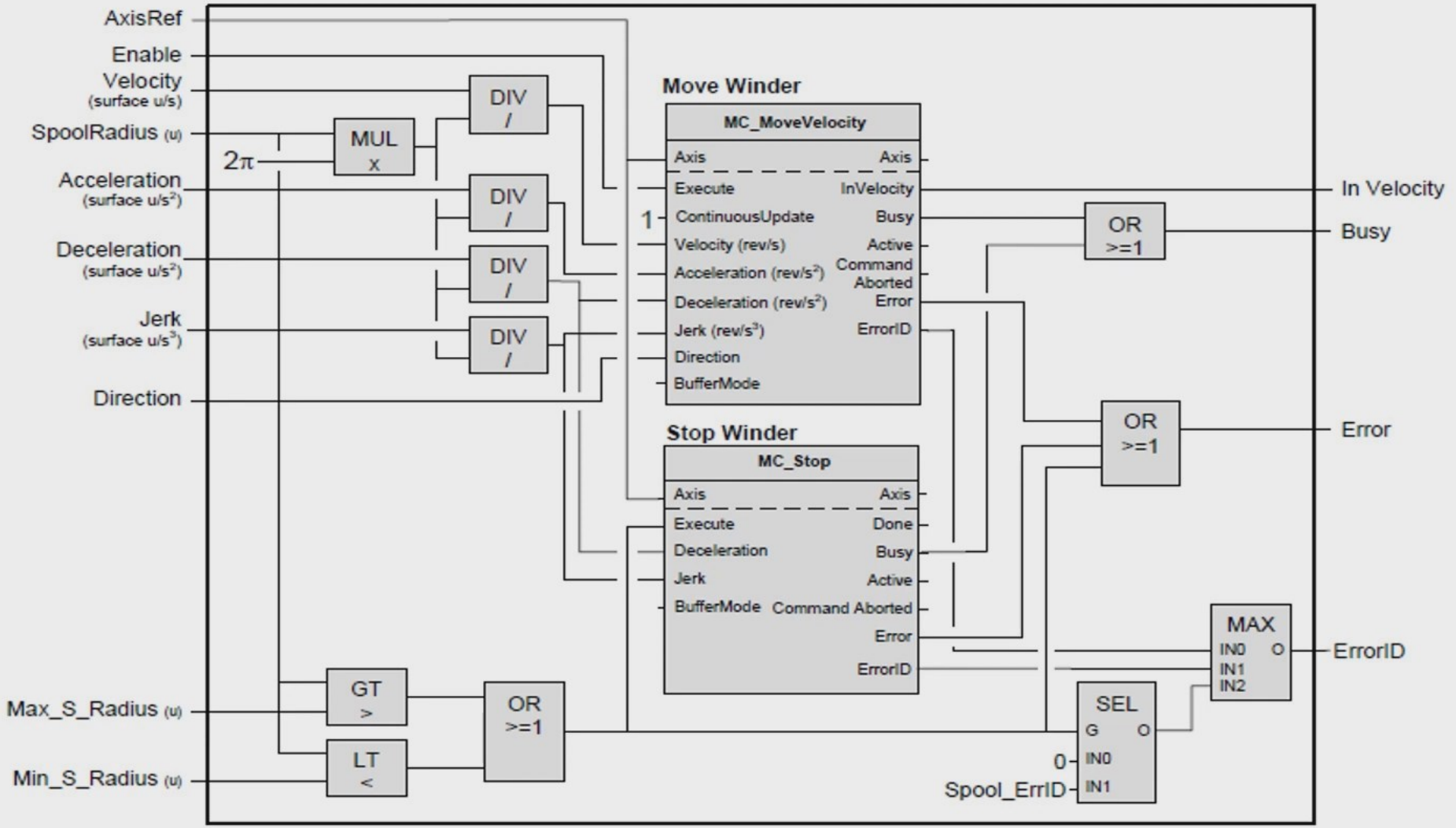


Graphical representation of FB

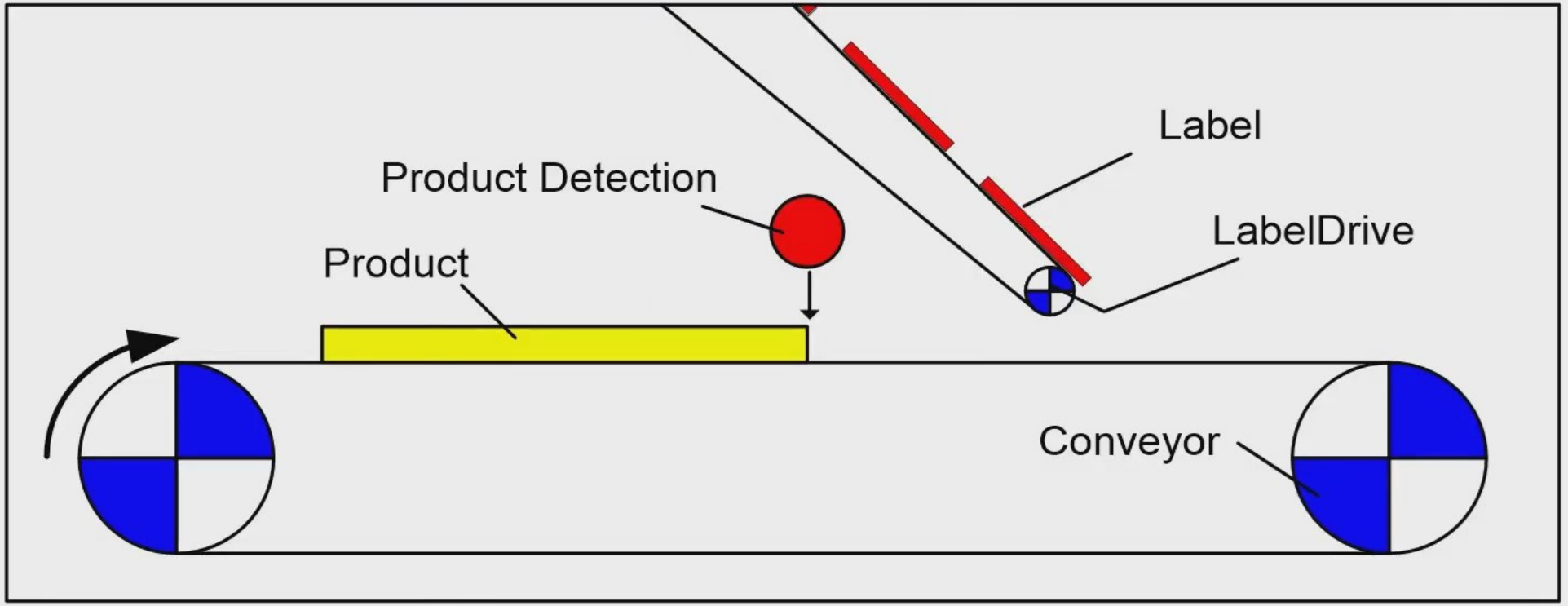


csv = constant surface velocity

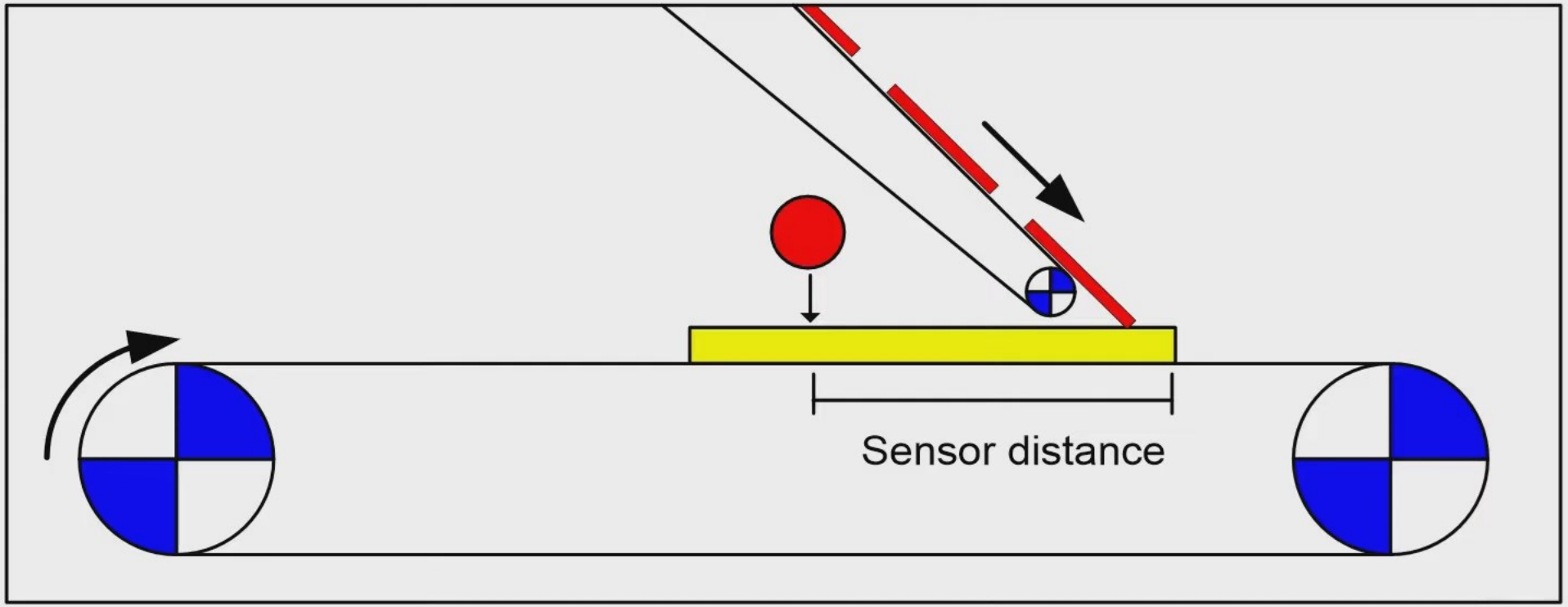
User Derived FB for Winding



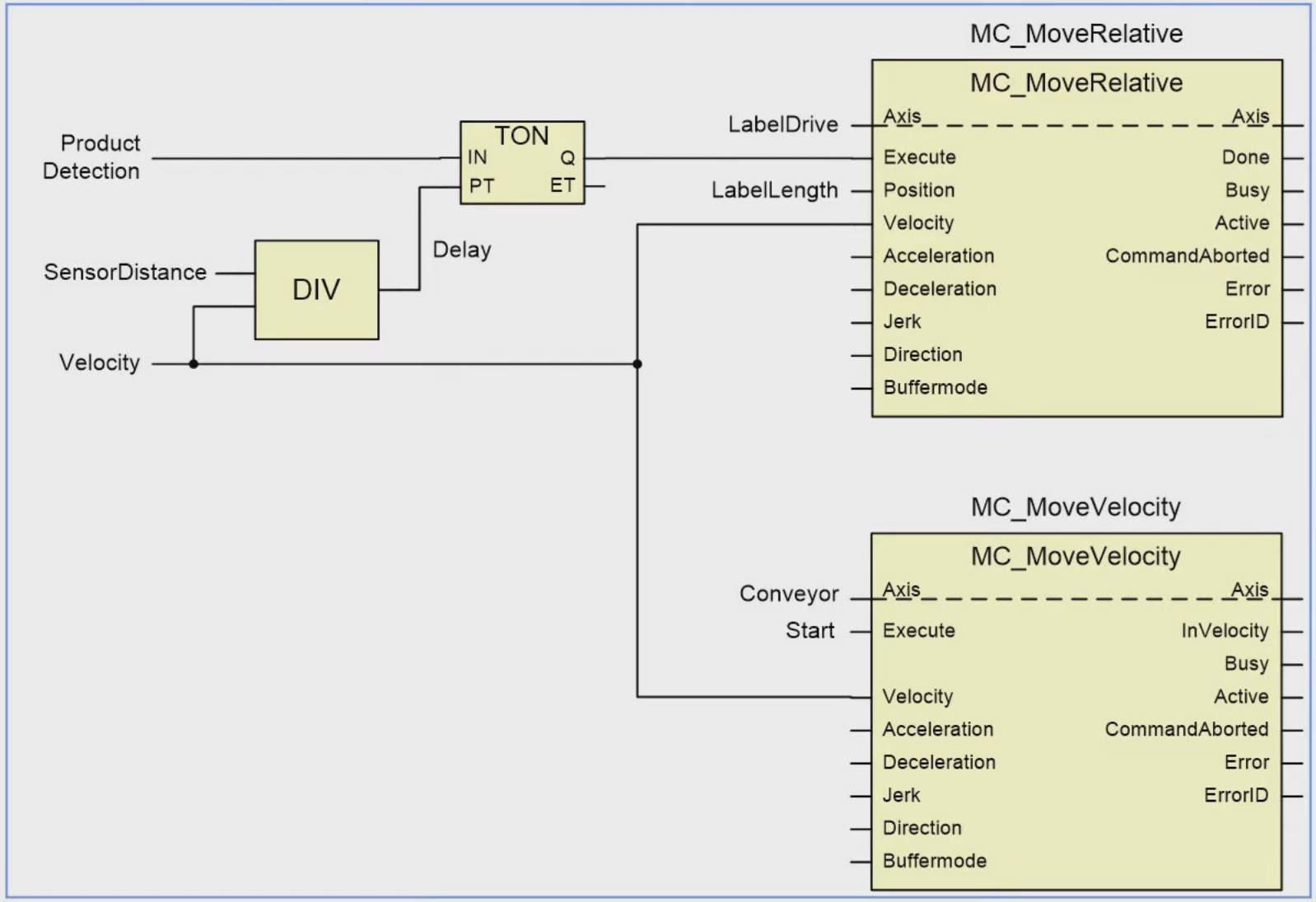
Labelling Example



Labelling Example



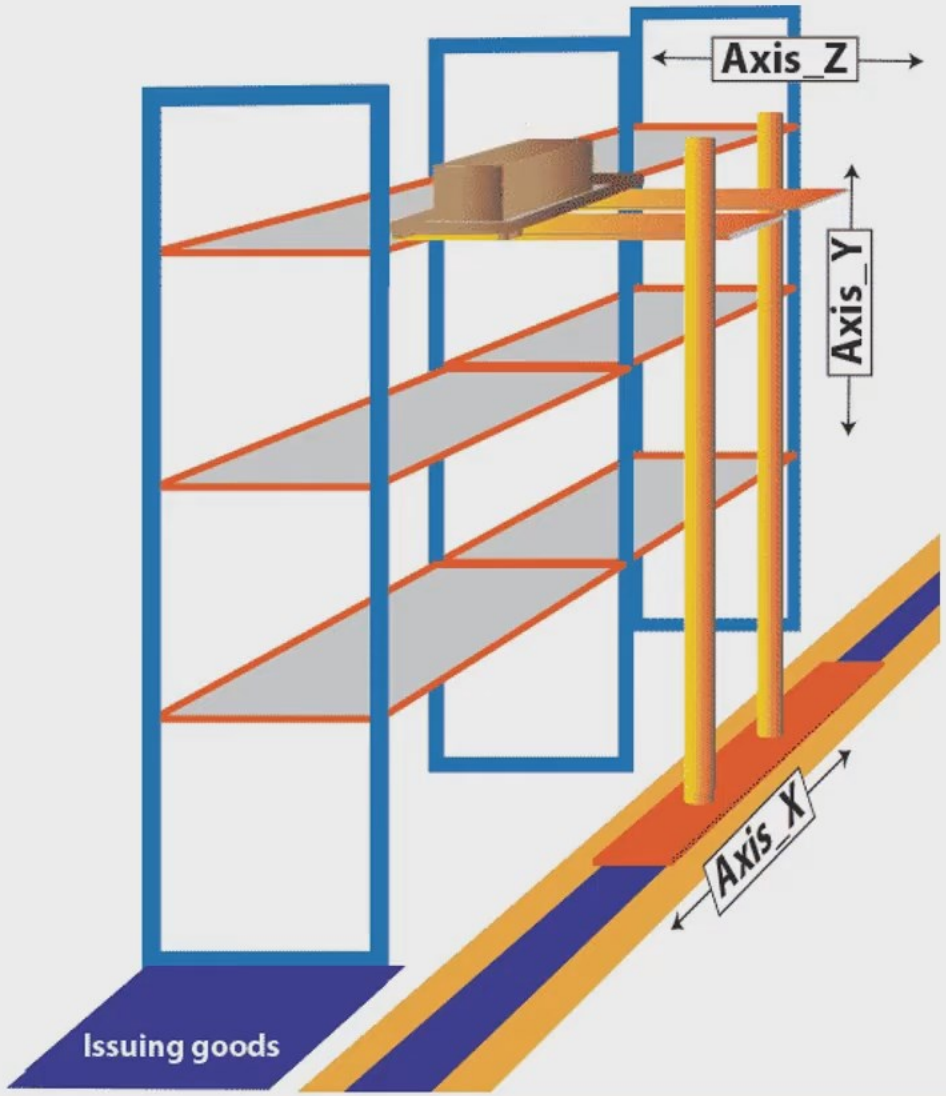
The labelling program



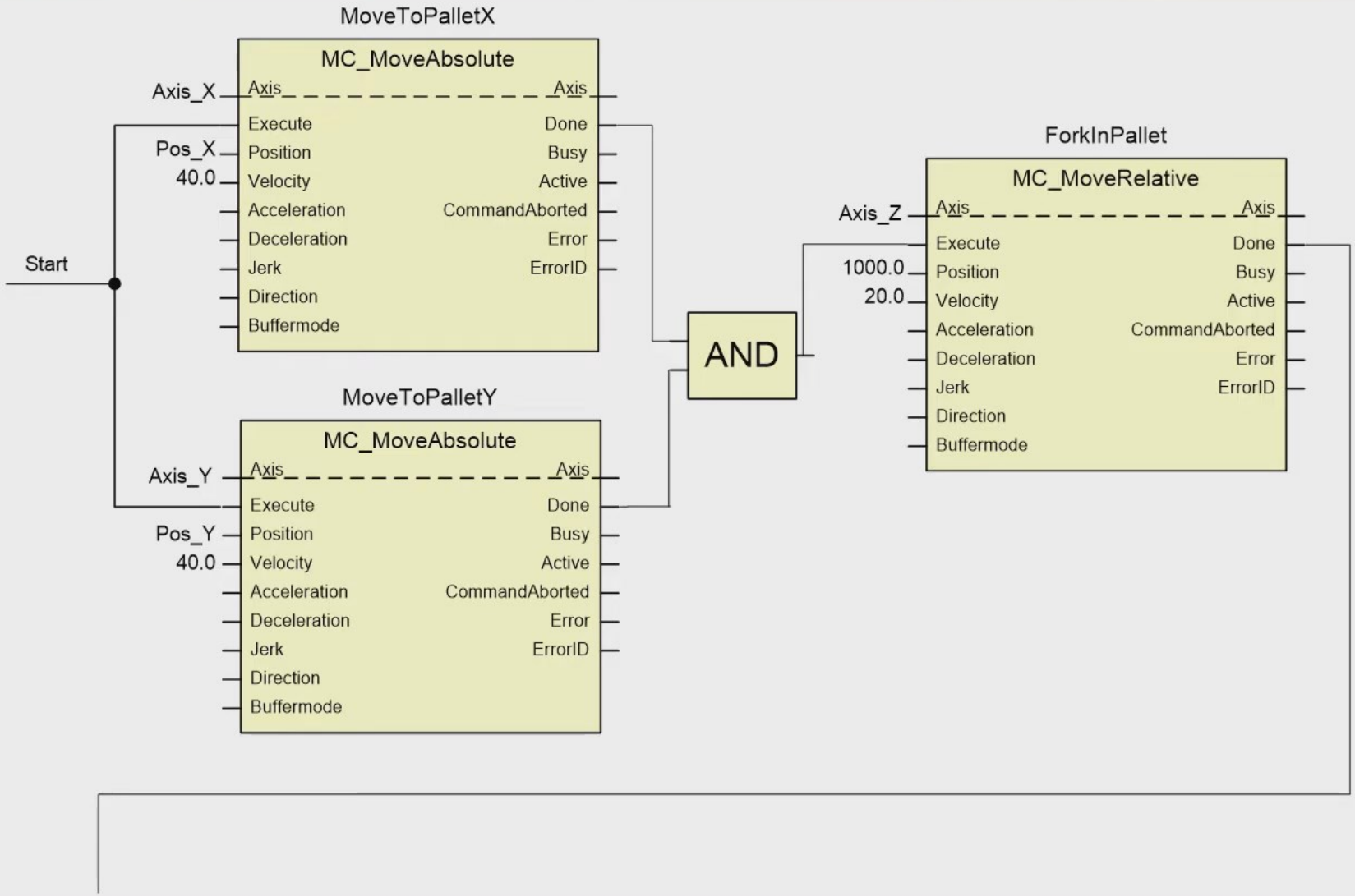
Possible improvements for more flexibility

- Compensate for the drift of the label position as a result of the sum of incremental errors.
- A fast touch-probe input to detect the start position of the product more precisely.
- An MC_CamIn or MC_GearInPos function to synchronize the label more exact on the product.
- If the product is smaller than the sensor distance a kind of FIFO for product tracking can be necessary.

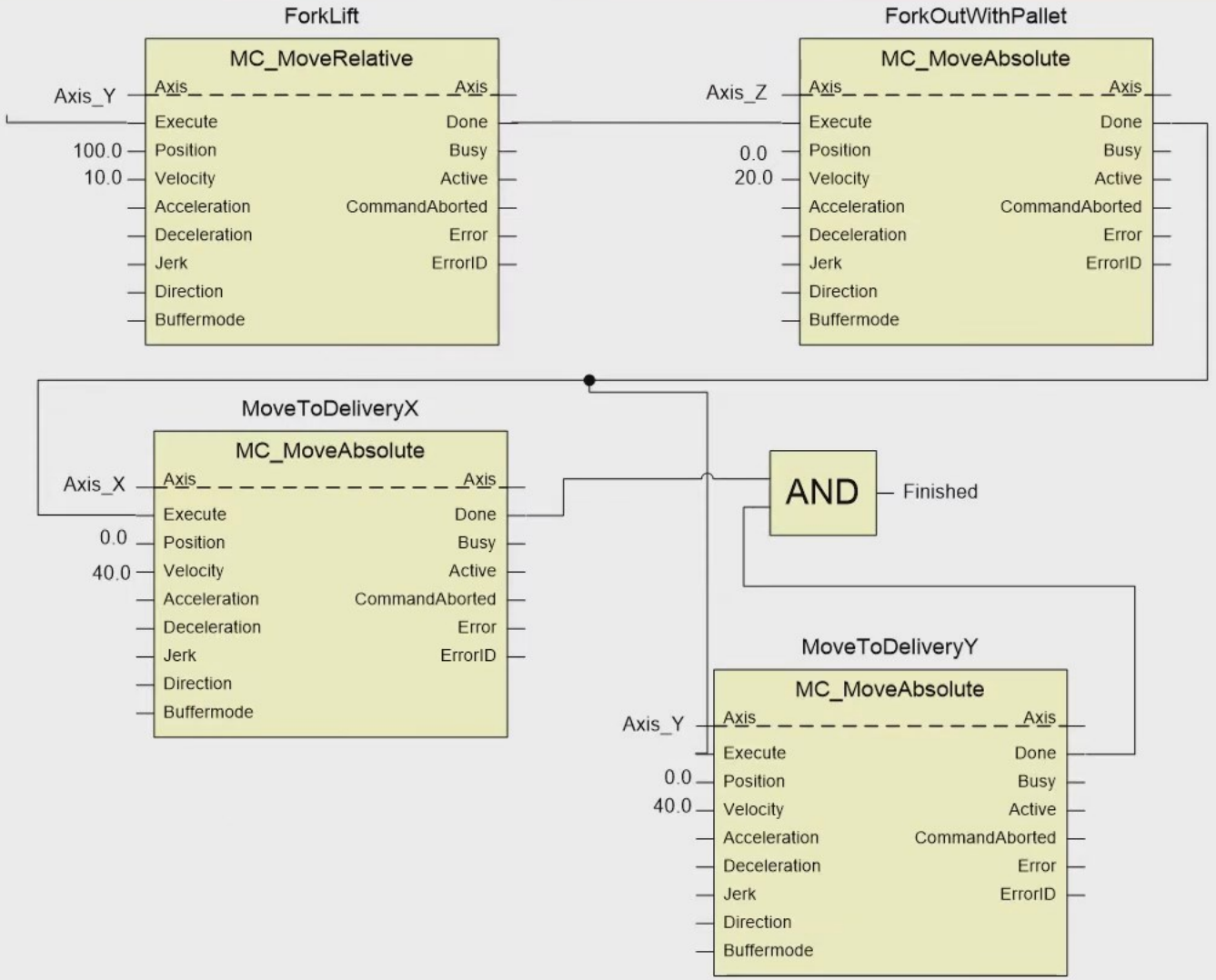
Warehousing Example



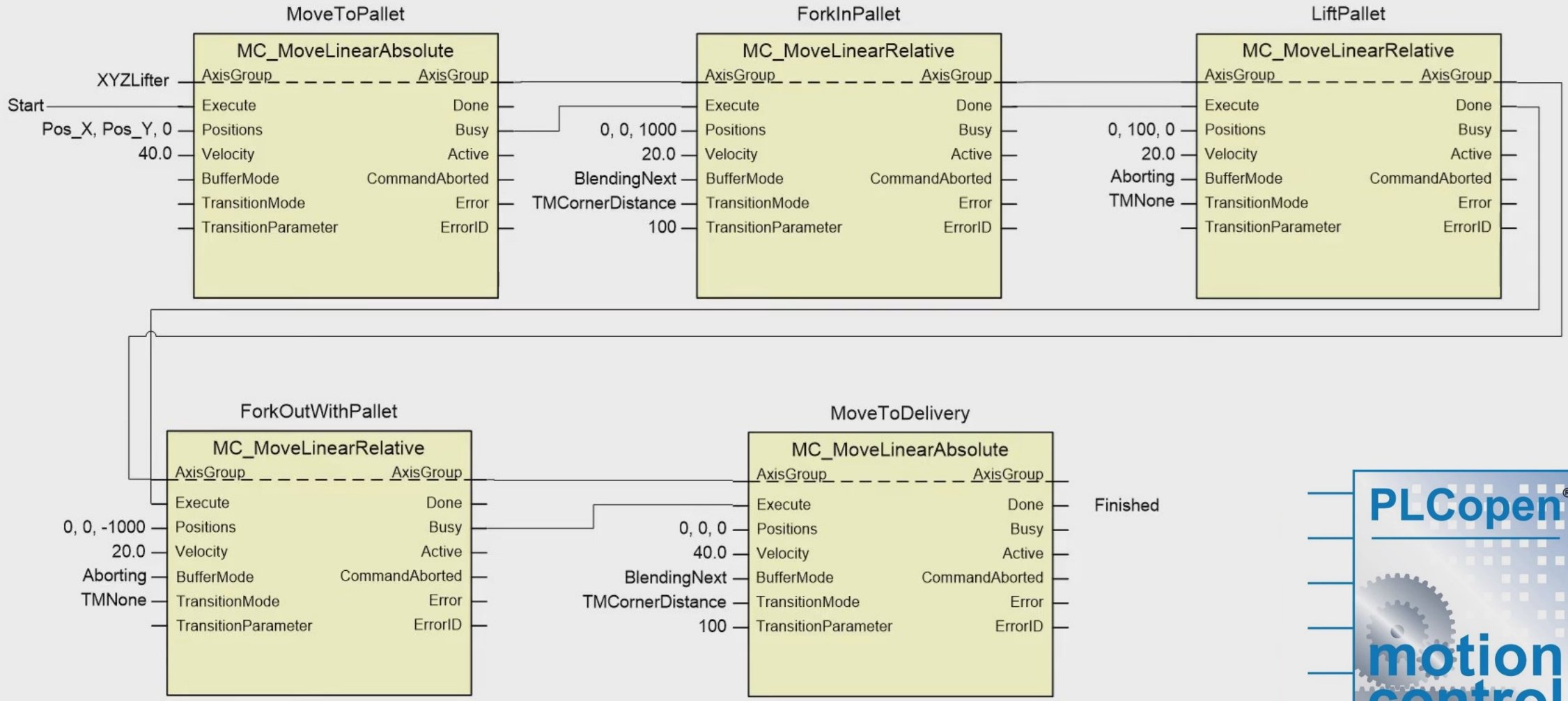
Application Program (1/2)



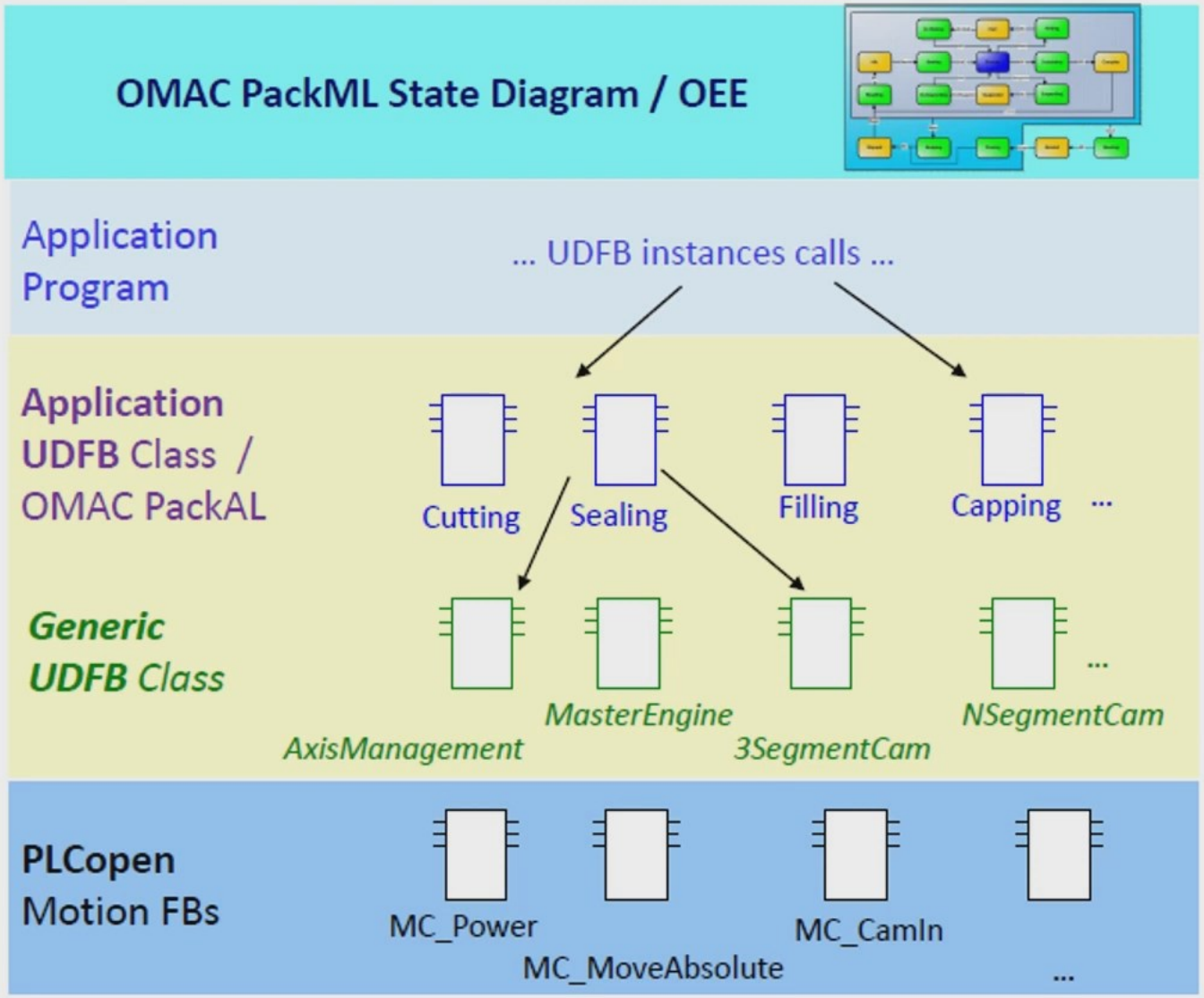
Application Program (2/2)



Alt. Application Program (with Part 4)



Layered approach of libraries



PLCopen – a suite of Specifications